Отчёт по лабораторной работе №5

Построение графиков

Умед Джамшедович Назармамадов

В работе рассмотрены средства построения графиков на языке Julia. Выполнены примеры использования пакета Plots и различных графических механизмов. Построены двумерные и трёхмерные графики, поверхности, линии уровня, векторные поля и гистограммы. Рассмотрены способы оформления графиков, отображения погрешностей и создания анимаций.

Содержание

1 Введение **8**
- 1.1 Актуальность . . . 8
- 1.2 Цель работы . . . 8
- 1.3 Задачи . . . 8
- 1.4 Объект и предмет исследования . . . 9
- 1.5 Методы исследования . . . 9
- 1.6 Схема выполнения работы . . . 9

2 Теоретическая часть **10**
- 2.1 Средства построения графиков . . . 10
- 2.2 Типы графиков . . . 11
- 2.3 Анимация . . . 11

3 Материалы и методы **12**

4 Выполнение лабораторной работы **13**
- 4.1 Основные пакеты для работы с графиками . . . 13
- 4.2 Опции при построении графика . . . 16
- 4.3 Точечные графики . . . 18
- 4.4 Аппроксимация . . . 21
- 4.5 Две оси ординат . . . 23
- 4.6 Полярные координаты . . . 23
- 4.7 Параметрические графики . . . 24
- 4.8 Графики поверхностей . . . 26
- 4.9 Линии уровня . . . 30
- 4.10 Векторные поля . . . 33
- 4.11 Анимация поверхности . . . 34
- 4.12 Построение гипоциклоиды . . . 36
- 4.13 Погрешности измерений . . . 38
- 4.14 Использование пакета Distributions . . . 42
- 4.15 Подграфики . . . 45

5 Самостоятельные задания **50**
- 5.1 Задание 1. Типы графиков функции sin(x) . . . 50
- 5.2 Задание 2. Стили линий . . . 51
- 5.3 Задание 3. Оформление графика . . . 54

5.4 Задание 4. Четыре способа изображения функции 56
5.5 Задание 5. Функции с различными масштабами 58
5.6 Задание 6. Данные с погрешностью измерения 59
5.7 Задание 7. Случайные точки на плоскости 60
5.8 Задание 8. Случайные точки в пространстве 61
5.9 Задание 9. Анимация синусоиды 62
5.10 Задание 10. Анимированные гипоциклоиды 63
5.11 Задание 11. Анимированные эпициклоиды 67

6 Результаты **70**

7 Обсуждение результатов **71**

8 Заключение **73**

Список литературы **74**

A Основные фрагменты кода **75**
A.1 Построение и сохранение графика 75
A.2 Приближение полиномом 76
A.3 Градиент функции 76
A.4 Общий принцип создания анимации 77

Список иллюстраций

4.1 Подготовка пакета Plots . . . 13
4.2 Первый график функции . . . 14
4.3 График функции с русскими подписями . . . 15
4.4 Сохранение графика Plotly в HTML . . . 15
4.5 Текстовое представление графика . . . 16
4.6 Сравнение синуса и приближения рядом Тейлора . . . 17
4.7 Оформление графиков синуса и ряда Тейлора . . . 18
4.8 Простой точечный график . . . 19
4.9 Точечный график с разными размерами маркеров . . . 20
4.10 Трёхмерный точечный график . . . 21
4.11 Приближение данных полиномом пятой степени . . . 22
4.12 Две случайные траектории с отдельными шкалами . . . 23
4.13 График в полярных координатах . . . 24
4.14 Параметрическая кривая на плоскости . . . 25
4.15 Параметрическая кривая в пространстве . . . 26
4.16 Поверхность функции двух переменных . . . 27
4.17 Каркас поверхности . . . 28
4.18 Поверхность с уменьшенным шагом сетки . . . 29
4.19 Поверхность с изменённым углом обзора . . . 30
4.20 Поверхность функции g(x,y) . . . 31
4.21 Линии уровня функции g(x,y) . . . 32
4.22 Линии уровня с цветной заливкой . . . 33
4.23 Линии уровня и градиентное поле . . . 34
4.24 Исходная волновая поверхность . . . 35
4.25 Формирование кадров анимации поверхности . . . 36
4.26 Частичное построение гипоциклоиды . . . 37
4.27 Вертикальные интервалы . . . 38
4.28 Горизонтальные интервалы . . . 39
4.29 Полоса неопределённости . . . 40
4.30 Интервалы по двум осям . . . 41
4.31 Асимметричные интервалы . . . 42
4.32 Гистограмма случайных возрастов . . . 43
4.33 Гистограмма нормального распределения . . . 44
4.34 Код построения гистограммы смеси распределений . . . 45
4.35 Четыре графика в одном столбце . . . 46
4.36 Автоматическое расположение четырёх графиков . . . 47

4.37 Объединение графиков разных типов 48
4.38 Сложный макет из одиннадцати подграфиков 49

5.1 Различные способы представления значений sin(x) 51
5.2 Сравнение стилей линий и предупреждение о неподдерживаемом стиле 53
5.3 Оформление графика функции с логарифмом 55
5.4 Четыре способа изображения функции 57
5.5 Две функции с общей вертикальной шкалой 58
5.6 Две функции с отдельными вертикальными шкалами 59
5.7 Учебные данные о нагревании воды 60
5.8 Сто случайных точек на плоскости 61
5.9 Сто случайных точек в пространстве 62
5.10 Создание анимации постепенного построения синусоиды 63
5.11 Гипоциклоида при k = 3 64
5.12 Гипоциклоида при k = 4 65
5.13 Гипоциклоида при k = 5/2 66
5.14 Гипоциклоида при k = 7/2 67
5.15 Эпициклоида при k = 3 68
5.16 Эпициклоида при k = 4 69

Список таблиц

2.1 Основные параметры оформления графиков 10

1 Введение

1.1 Актуальность

Графическое представление данных позволяет наглядно исследовать поведение функций, сравнивать результаты вычислений и выявлять закономерности. Средства визуализации применяются при статистическом анализе, математическом моделировании и обработке экспериментальных данных.

В языке Julia доступны инструменты для построения графиков различных типов, настройки их оформления и создания анимаций.

1.2 Цель работы

Освоить синтаксис языка Julia для построения графиков.

1.3 Задачи

1. Повторить примеры из раздела 5.2 методических указаний.
2. Дополнить графики названиями, подписями осей и легендами.
3. Изучить построение точечных графиков, поверхностей, линий уровня и векторных полей.
4. Рассмотреть отображение погрешностей измерений и распределений случайных величин.
5. Изучить объединение нескольких графиков в одном окне.

6. Выполнить задания для самостоятельной работы из раздела 5.4.
7. Создать анимации синусоиды и кривых, связанных с качением окружностей.

1.4 Объект и предмет исследования

Объект исследования: средства графического представления данных в Julia.

Предмет исследования: команды построения и оформления графиков, отображения случайных данных и создания анимаций.

1.5 Методы исследования

В работе использовались вычислительные эксперименты в JupyterLab, построение графиков функций, генерация случайных данных и визуальное сравнение результатов.

1.6 Схема выполнения работы

Работа выполнялась в следующем порядке:

1. Подготовка рабочего окружения и подключение пакетов.
2. Последовательное выполнение примеров методических указаний.
3. Построение графиков с дополнительным оформлением.
4. Выполнение самостоятельных заданий.
5. Сохранение результатов и подготовка отчёта.

2 Теоретическая часть

2.1 Средства построения графиков

Пакет `Plots` предоставляет общий интерфейс для построения графиков. Для непосредственного отображения используются графические механизмы, в том числе GR, PyPlot и Plotly.

Команда `plot()` создаёт новый график. Команда `plot!()` добавляет данные к существующему графику.

Основные параметры оформления приведены в Таблица 2.1.

Таблица 2.1: Основные параметры оформления графиков

Параметр	Назначение
`title`	Название графика
`xlabel`, `ylabel`, `zlabel`	Подписи координатных осей
`label`	Подпись серии данных в легенде
`color`	Цвет
`linewidth`	Толщина линии
`linestyle`	Стиль линии
`markersize`	Размер маркеров
`alpha`	Прозрачность
`layout`	Расположение подграфиков
`size`	Размер графического окна

Для сохранения рисунка используется команда `savefig()`.

2.2 Типы графиков

Линейный график показывает изменение величины и соединяет точки отрезками. Точечный график отображает отдельные наблюдения без соединения линиями.

Гистограмма показывает количество наблюдений, попавших в заданные интервалы.

Поверхность представляет зависимость одной величины от двух переменных. Линии уровня соединяют точки, в которых функция принимает одинаковое значение.

Векторное поле изображается стрелками. Для функции двух переменных её градиент имеет вид

$$\nabla h(x, y) = \left(\frac{\partial h}{\partial x}, \frac{\partial h}{\partial y} \right).$$

Градиент направлен в сторону наиболее быстрого увеличения функции.

2.3 Анимация

Анимация представляет последовательность кадров. При изменении параметров функции или количества отображаемых точек на каждом кадре получается движущееся изображение.

В пакете `Plots` кадры можно сформировать с помощью `@animate`, а затем сохранить командой `gif()`.

3 Материалы и методы

Работа выполнялась в среде WSL Ubuntu с использованием Julia и JupyterLab с ядром IJulia.

Применялись следующие пакеты:

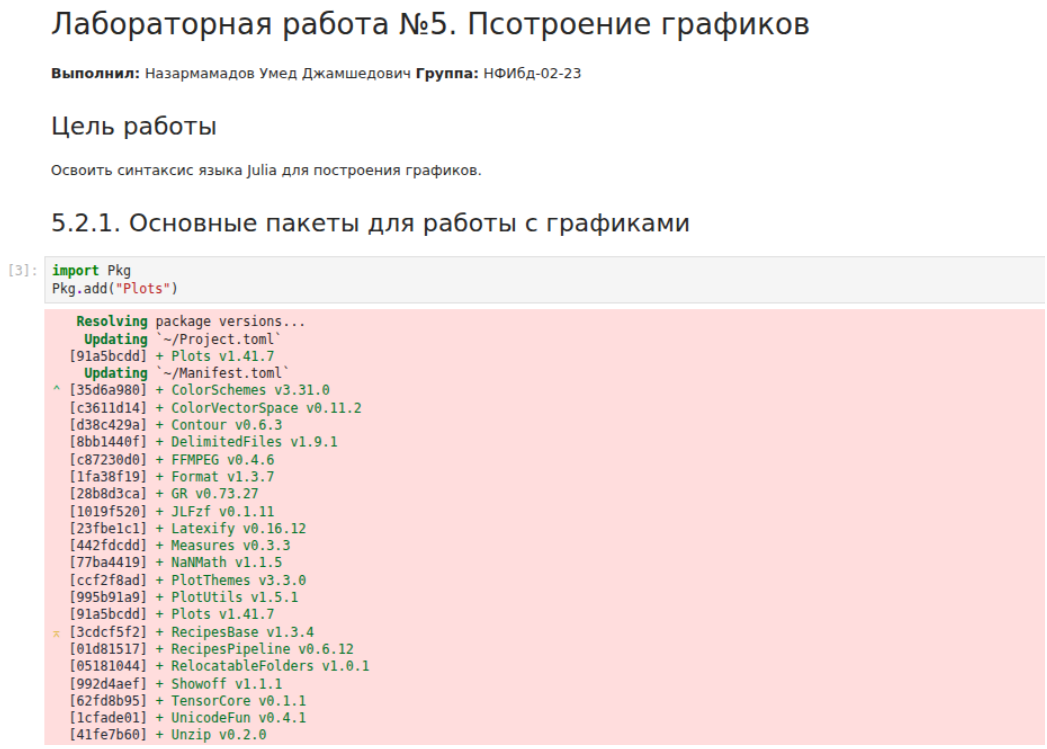
- `Plots` для построения графиков;
- `PyPlot` для графического отображения;
- `UnicodePlots` для текстового представления графика;
- `Statistics` для расчёта средних значений и стандартных отклонений;
- `Distributions` для генерации данных с заданным распределением.

Результаты выполнения фиксировались скриншотами. При случайной генерации данных конкретное расположение точек и высота столбцов могут изменяться при повторном запуске.

4 Выполнение лабораторной работы

4.1 Основные пакеты для работы с графиками

В начале работы был подключён менеджер пакетов `Pkg` и выполнена установка `Plots`. Подготовка пакета представлена на Рисунок 4.1.



Лабораторная работа №5. Псотроение графиков

Выполнил: Назармамадов Умед Джамшедович **Группа:** НФИбд-02-23

Цель работы

Освоить синтаксис языка Julia для построения графиков.

5.2.1. Основные пакеты для работы с графиками

```
[3]: import Pkg
     Pkg.add("Plots")
```

```
   Resolving package versions...
    Updating `~/Project.toml`
  [91a5bcdd] + Plots v1.41.7
    Updating `~/Manifest.toml`
^ [35d6a980] + ColorSchemes v3.31.0
  [c3611d14] + ColorVectorSpace v0.11.2
  [d38c429a] + Contour v0.6.3
  [8bb1440f] + DelimitedFiles v1.9.1
  [c87230d0] + FFMPEG v0.4.6
  [1fa38f19] + Format v1.3.7
  [28b8d3ca] + GR v0.73.27
  [1019f520] + JLFzf v0.1.11
  [23fbe1c1] + Latexify v0.16.12
  [442fdcdd] + Measures v0.3.3
  [77ba4419] + NaNMath v1.1.5
  [ccf2f8ad] + PlotThemes v3.3.0
  [995b91a9] + PlotUtils v1.5.1
  [91a5bcdd] + Plots v1.41.7
⌅ [3cdcf5f2] + RecipesBase v1.3.4
  [01d81517] + RecipesPipeline v0.6.12
  [05181044] + RelocatableFolders v1.0.1
  [992d4aef] + Showoff v1.1.1
  [62fd8b95] + TensorCore v0.1.1
  [1cfade01] + UnicodeFun v0.4.1
  [41fe7b60] + Unzip v0.2.0
```

Рисунок 4.1: Подготовка пакета Plots

Для первого графика использовалась функция

$$f(x) = (3x^2 + 6x - 9)e^{-0.3x}.$$

Значения аргумента задавались на отрезке от −5 до 10 с шагом 0.1. Значения функции вычислялись поэлементно с помощью `f.(x)`.

Сначала график был построен с английскими подписями осей и легендой (Рисунок 4.2).

```
[6]: plot(
         x, y,
         title="Function graph",
         xlabel="x",
         ylabel="f(x)",
         label="(3x^2 + 6x - 9) exp(-0.3x)",
         color=:blue,
         linewidth=2,
         grid=true,
         size(800, 500)
     )
```

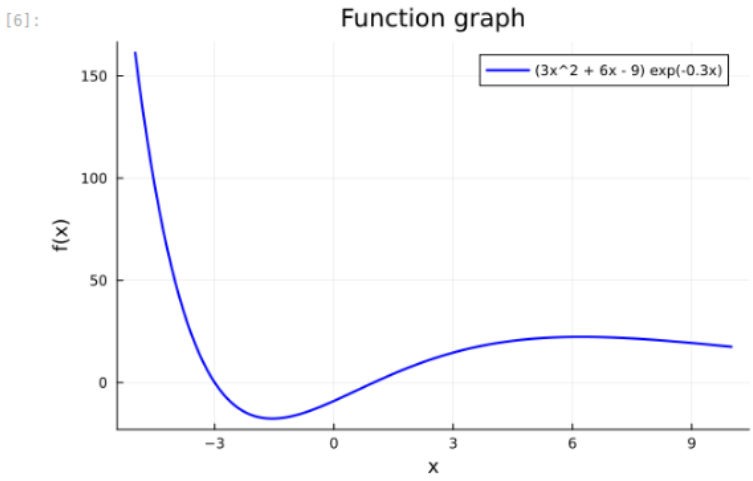


Рисунок 4.2: Первый график функции

Затем был построен вариант с русскими подписями (Рисунок 4.3). Форма кривой сохранилась.

```
[10]: plot(
          x, y,
          title="Простая кривая",
          xlabel="Переменная x",
          ylabel="Переменная y",
          label="f(x)",
          color=:blue,
          linewidth=2,
          grid=true,
          size=(800, 500)
      )
```

[10]:

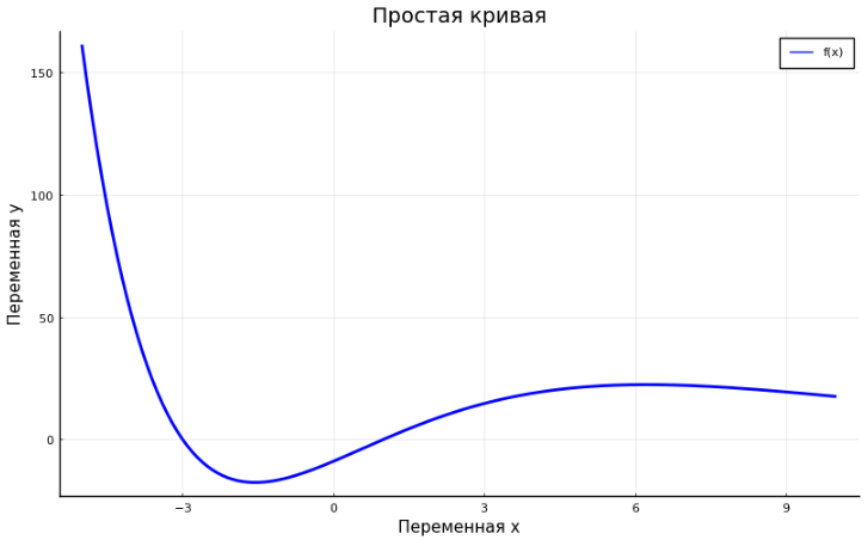


Рисунок 4.3: График функции с русскими подписями

При использовании Plotly график не отобразился непосредственно в выводе ноутбука. Поэтому в код была добавлена команда сохранения в HTML. На Рисунок 4.4 показан результат сохранения файла.

```
[17]: plotly()

      p = plot(
          x, y,
          title="График функции: Plotly",
          xlabel="x",
          ylabel="f(x)",
          label="f(x)",
          color=:blue,
          linewidth=2,
          size=(800, 500)
      )
      savefig(p, "plotly_graph.html")
```

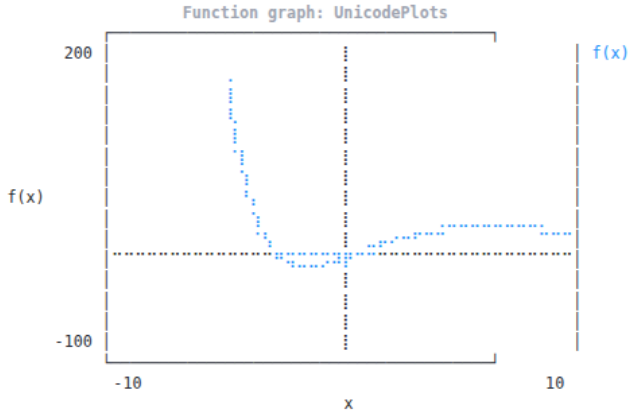
```
[17]: "/home/umedn/work/study/2026-2/2026-2==computer-practice/2026-2--study--computer-practice/labs/lab05/project/plotly_graph.html"
```

Рисунок 4.4: Сохранение графика Plotly в HTML

Для текстового представления использовалась функция `UnicodePlots.lineplot()`. Результат был выведен через `show` с форматом `text/plain` (Рисунок 4.5).

```
[29]: import UnicodePlots

p_text = UnicodePlots.lineplot(
    x, y,
    title="Function graph: UnicodePlots",
    xlabel="x",
    ylabel="f(x)",
    name="f(x)",
    color=:blue
)
show(stdout, MIME"text/plain"(), p_text)
```




```
[29]: (20, 60)
```

Рисунок 4.5: Текстовое представление графика

4.2 Опции при построении графика

Для исследования оформления сравнивались функция $\sin(x)$ и сумма первых пяти членов её ряда Тейлора:

$$S_9(x) = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \frac{x^9}{9!}.$$

Обе функции были построены на отрезке $[-5, 5]$. При совместном отображении видно, что около нуля кривые практически совпадают, а при удалении от нуля различие увеличивается (Рисунок 4.6).

[33]:
```
plot(
    sin_theor, -5, 5,
    title="Сравнение синуса и ряда Тейлора",
    xlabel="x",
    ylabel="y",
    label="sin(x)",
    color=:blue,
    linewidth=2
)

plot!(
    sin_taylor, -5, 5,
    label="Ряд Тейлора",
    color=:blue,
    linewidth=2
)
```

[33]:

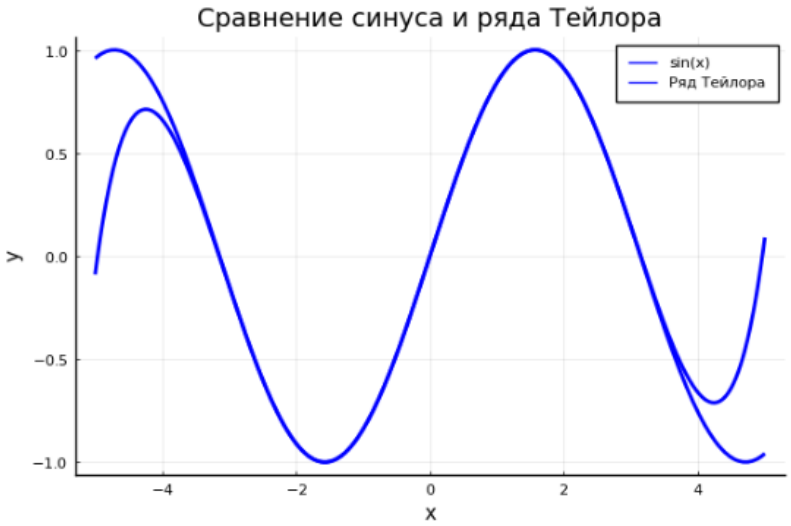


Рисунок 4.6: Сравнение синуса и приближения рядом Тейлора

Затем были настроены толщина, прозрачность и стиль линий, цвет фона, заливка, шрифты и отметки на осях. Результат представлен на Рисунок 4.7.

[34]:
```
p_taylor = plot(
    sin_taylor, -5, 5,
    label="sin(x), ряд Тейлора",
    linecolor=:blue,
    linealpha=0.3,
    linewidth=6,
    linestyle=:solid,

    size=(800, 500),

    xticks=-5:0.5:5,
    yticks=-1:0.1:1,
    xtickfont=font(12, "DejaVu Serif"),
    ytickfont=font(12, "DejaVu Serif"),

    xlabel="x",
    ylabel="y",
    title="Разложение синуса в ряд Тейлора",
    xrotation=45,

    fillrange=0,
    fillalpha=0.5,
    fillcolor=:lightgoldenrod,

    background_color=:ivory,
    legend=:topleft
)

plot!(
    p_taylor,
    sin_theor, -5, 5,
    label="sin(x), точное значение",
    linecolor=:black,
    linealpha=1,
    linewidth=2,
    linestyle=:dash
)
```

[34]:



Рисунок 4.7: Оформление графиков синуса и ряда Тейлора

4.3 Точечные графики

Сначала были созданы десять случайных значений и построен простой точечный график (Рисунок 4.8).

5.2.3. Точечный график

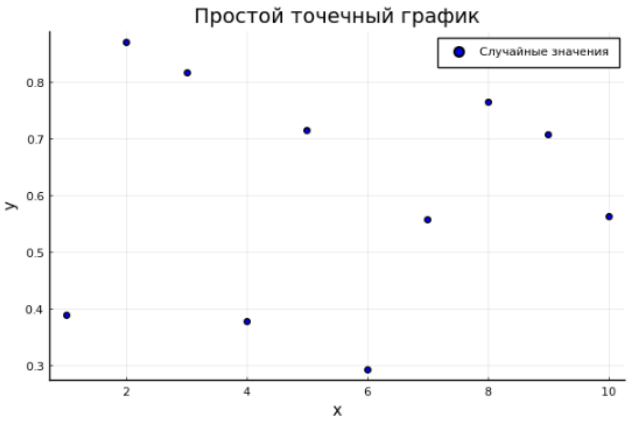
5.2.3.1. Простой точечный график

```
[36]: pyplot()

x = range(1, 10, length=10)
y = rand(10)

plot(
    x, y,
    seriestype=:scatter,
    title="Простой точечный график",
    xlabel="x",
    ylabel="y",
    label="Случайные значения",
    color=:blue,
    markesize=6
)
```

[36]:




```
sys:1: UserWarning: No data for colormapping provided via 'c'. Parameters 'vmin', 'vmax' will be ignored
```

Рисунок 4.8: Простой точечный график

Затем были созданы 50 точек со случайными координатами и размерами маркеров (Рисунок 4.9).

5.2.3.2. Точечный график с разными размерами маркеров

[37]:
```
n = 50

x = rand(n)
y = rand(n)
ms = rand(n) * 30

scatter(
    x, y,
    markersize=ms,
    title="Точки разного размера",
    xlabel="x",
    ylabel="y",
    label="Случайные точки",
    color=:blue,
    alpha=0.6
)
```

[37]:

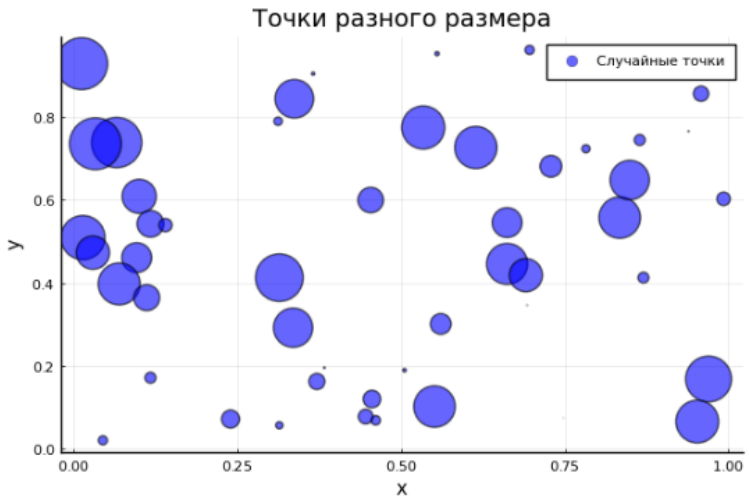


Рисунок 4.9: Точечный график с разными размерами маркеров

После добавления третьей координаты был построен трёхмерный точечный график (Рисунок 4.10).

5.2.3.3. Трехмерный точечный график с разными размерами маркеров

[38]:
```
n = 50

x = rand(n)
y = rand(n)
z = rand(n)
ms = rand(n) * 30

scatter(
    x, y, z,
    markersize=ms,
    title="Трехмерный точечный график",
    xlabel="x",
    ylabel="y",
    zlabel="z",
    label="Случайные точки",
    color=:blue,
    alpha=0.6,
    size=(800, 600)
)
```

[38]:

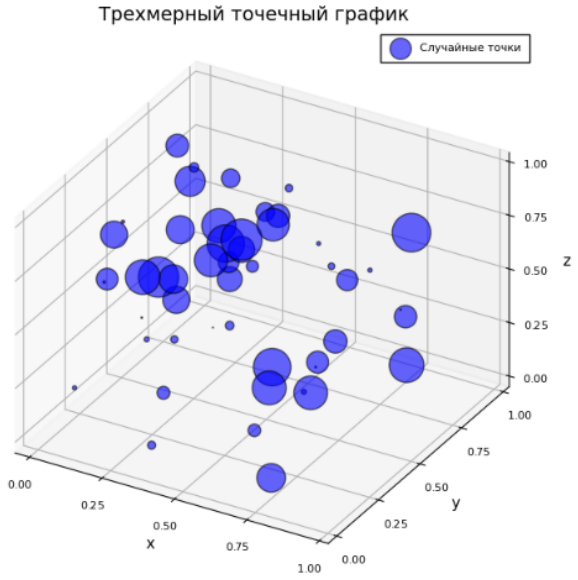


Рисунок 4.10: Трёхмерный точечный график

4.4 Аппроксимация

Для исследования аппроксимации были сформированы значения экспоненциальной функции со случайными отклонениями.

Приближение выполнялось полиномом пятой степени:

$$p(x) = c_1 + c_2 x + c_3 x^2 + c_4 x^3 + c_5 x^4 + c_6 x^5.$$

Для определения коэффициентов была сформирована матрица

$$A = \begin{pmatrix} 1 & x_1 & x_1^2 & x_1^3 & x_1^4 & x_1^5 \\ 1 & x_2 & x_2^2 & x_2^3 & x_2^4 & x_2^5 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \end{pmatrix}.$$

Команда `c = A \ y` определяет коэффициенты методом наименьших квадратов. Полученный полином был добавлен к исходным точкам (Рисунок 4.11).

```
[40]: A = [ones(1000) x x.^2 x.^3 x.^4 x.^5]

      c = A \ y
```

```
[40]: 6-element Vector{Float64}:
       -1448.1431170130238
        2664.85894479813
       -2507.7937875595458
         925.7713908784722
        -140.31694464449677
           7.605677682083299
```

```
[41]: y_fit = c[1] .+ c[2] .* x .+ c[3] .* x.^2 .+
              c[4] .* x.^3 .+ c[5] .* x.^4 .+ c[6] .* x.^5

      plot!(
          p_approx,
          x, y_fit,
          linewidth=3,
          color=:red,
          label="Полином пятой степени",
          title="Приближение данных полиномом"
      )
```

[41]:

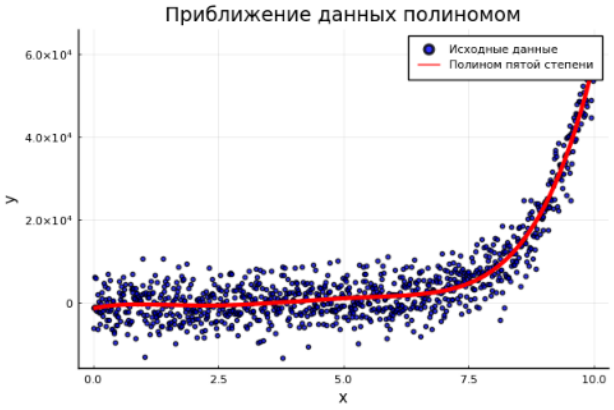


Рисунок 4.11: Приближение данных полиномом пятой степени

Красная кривая передаёт общую тенденцию роста данных, не проходя через каждую случайную точку.

4.5 Две оси ординат

Были созданы две случайные последовательности с различными масштабами значений. Для второй последовательности использовалось умножение на 10.

С помощью `twinx()` к графику была добавлена правая вертикальная ось (Рисунок 4.12).

```
[44]: plot!(
          twinx(),
          y2,
          ylabel="y2",
          label="Вторая траектория",
          color=:red,
          legend=:bottomright,
          grid=false
      )
```



Рисунок 4.12: Две случайные траектории с отдельными шкалами

Синяя линия соответствует левой шкале, красная правой.

4.6 Полярные координаты

В полярной системе координат была построена функция

$$r(\theta) = 1 + \cos(\theta)\sin^2(\theta), \qquad 0 \leq \theta \leq 2\pi.$$

Для выбора полярной системы использовался параметр `proj=:polar`. Результат представлен на Рисунок 4.13.

5.2.6. Полярные координаты

[46]:
```
pyplot()

# Расстояние от центра в зависимости от угла
r(θ) = 1 + cos(θ) * sin(θ)^2

# Углы от 0 до полного оборота
θ = range(0, stop=2π, length=50)

p_polar = plot(
    θ, r.(θ),
    proj=:polar,
    ylims=(0, 1.5),
    title="График в полярных координатах",
    label="r(θ) = 1 + cos(θ) sin²(θ)",
    linewidth=2,
    color=:blue,
    size=(700, 600)
)
```

[46]:



Рисунок 4.13: График в полярных координатах

4.7 Параметрические графики

На плоскости была построена кривая

$$x(t) = \sin(t), \qquad y(t) = \sin(2t), \qquad 0 \leq t \leq 2\pi.$$

Полученная кривая имеет форму восьмёрки (Рисунок 4.14).

5.2.7. Параметрический график

5.2.7.1. Кривая на плоскости

[48]:
```
pyplot()

t = range(0, stop=2π, length=500)

x = sin.(t)
y = sin.(2 .* t)

p_param2d = plot(
    x, y,
    title="Параметрическая кривая на плоскости",
    xlabel="x",
    ylabel="y",
    label="x = sin(t), y = sin(2t)",
    color=:blue,
    linewidth=2,
    fillrange=0,
    fillcolor=:orange,
    fillalpha=0.4,
    aspect_ratio=:equal
)
```

[48]:

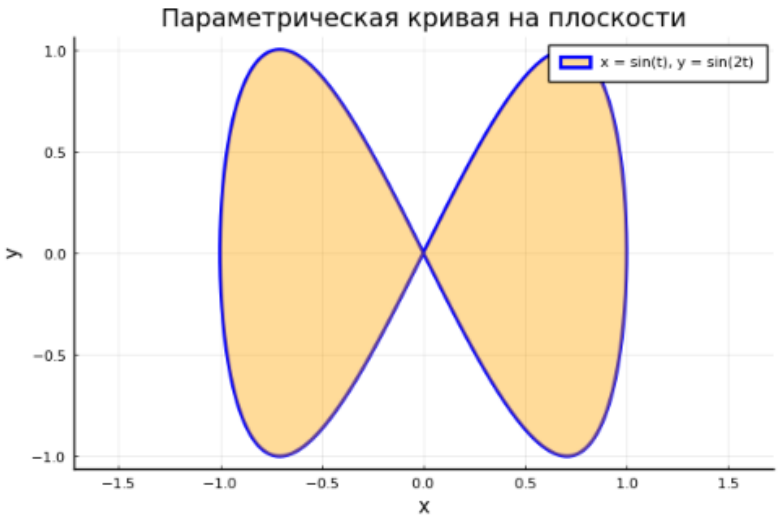


Рисунок 4.14: Параметрическая кривая на плоскости

В пространстве использовались зависимости

$$x(t) = \cos(t), \qquad y(t) = \sin(t), \qquad z(t) = \sin(5t), \qquad 0 \leq t \leq 10.$$

Результат показан на Рисунок 4.15.

5.2.7.2. Кривая в пространстве

[50]:
```
t = range(0, stop=10, length=1000)

x = cos.(t)
y = sin.(t)
z = sin.(5 .* t)

p_param3d = plot(
    x, y, z,
    title="Параметрическая кривая в пространстве",
    xlabel="x",
    ylabel="y",
    zlabel="z",
    label="Пространственная кривая",
    color=:blue,
    linewidth=2,
    size=(800, 600)
)
```

[50]:

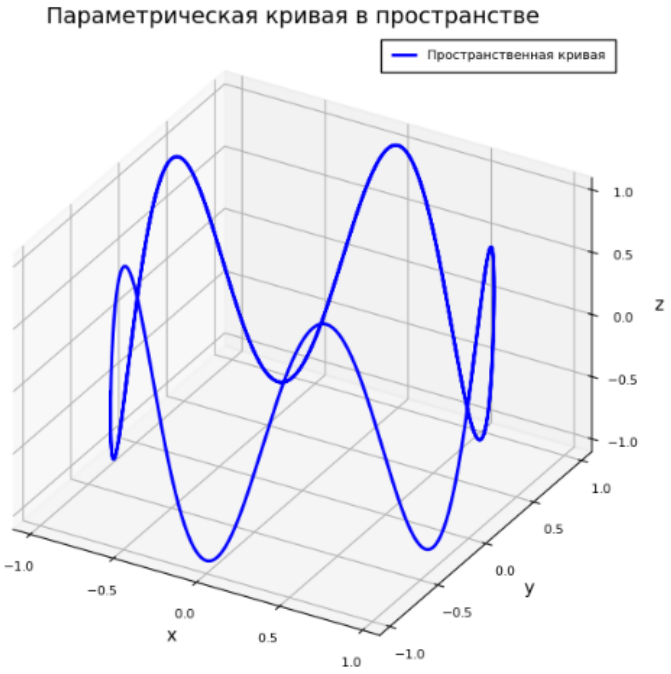


Рисунок 4.15: Параметрическая кривая в пространстве

4.8 Графики поверхностей

Была построена поверхность

$$z = x^2 + y^2.$$

При использовании `surface()` получен параболоид (Рисунок 4.16).

5.2.8. График поверхности

[52]:
```
pyplot()

f_surface(x, y) = x^2 + y^2

x = -10:10
y = -10:10

p_surface = surface(
    x, y, f_surface,
    title="Поверхность z = x² + y²",
    xlabel="x",
    ylabel="y",
    zlabel="z",
    color=:viridis,
    legend=false,
    size=(800, 600)
)
```

[52]:

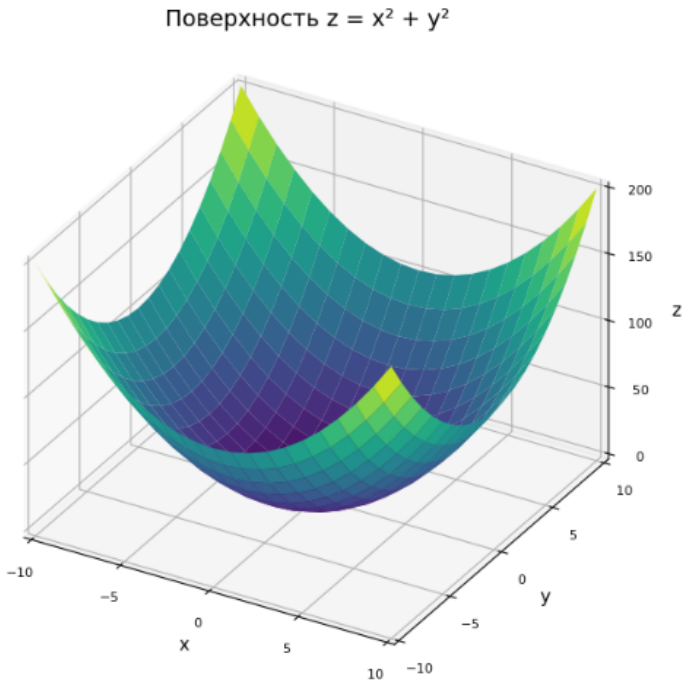


Рисунок 4.16: Поверхность функции двух переменных

Также был построен каркас поверхности с помощью `wireframe()` (Рисунок 4.17).

Каркас поверхности

[55]:
```
p_wire = wireframe(
    -10:10, -10:10, f_surface,
    title="Каркас поверхности z = x² + y²",
    xlabel="x",
    ylabel="y",
    zlabel="z",
    legend=false,
    size=(800, 600)
)
```

[55]:

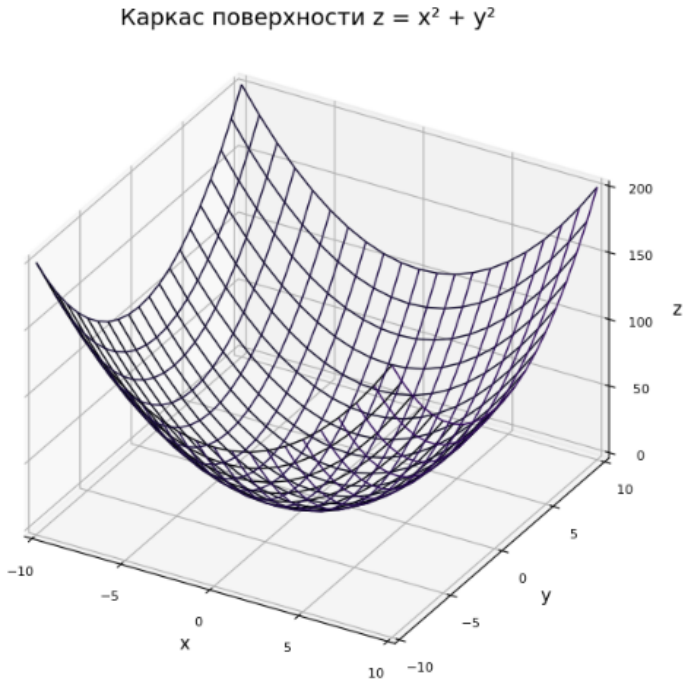


Рисунок 4.17: Каркас поверхности

Уменьшение шага сетки с 1 до 0.1 позволило получить более гладкое изображение (Рисунок 4.18).

Поверхность с мелким шагом сетки

[56]:
```
x = -10:0.1:10
y = -10:0.1:10

p_smooth = surface(
    x, y, f_surface,
    title="Поверхность с шагом сетки 0.1",
    xlabel="x",
    ylabel="y",
    zlabel="z",
    color=:viridis,
    legend=false,
    size=(800, 600)
)
```

[56]:

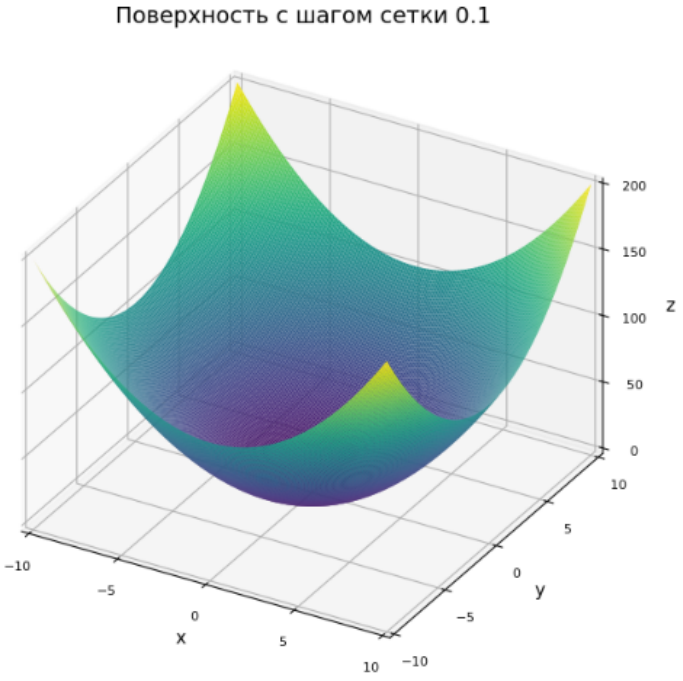


Рисунок 4.18: Поверхность с уменьшенным шагом сетки

Для функции

$$z = xy - x - y + 1$$

были изменены цветовая палитра и направление обзора с помощью параметра `camera` (Рисунок 4.19).

Изменение угла обзора поверхности

[57]:
```
x = range(-2, stop=2, length=100)
y = range(sqrt(2), stop=2, length=100)

f_view(x, y) = x*y - x - y + 1

p_camera = surface(
    x, y, f_view,
    title="Поверхность с изменённым углом обзора",
    xlabel="x",
    ylabel="y",
    zlabel="z",
    color=cgrad([:red, :blue]),
    camera=(-30, 30),
    legend=false,
    size=(800, 600)
)
```

[57]:

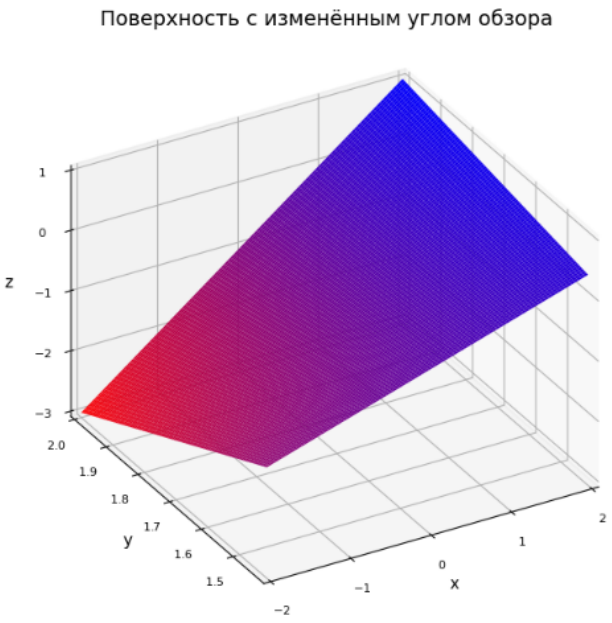


Рисунок 4.19: Поверхность с изменённым углом обзора

4.9 Линии уровня

Рассматривалась функция

$$g(x, y) = (3x + y^2)|\sin(x) + \cos(y)|.$$

Сначала была построена её поверхность (Рисунок 4.20).

5.2.9. Линии уровня

[58]:
```
pyplot()

x = 1:0.5:20
y = 1:0.5:10

g(x, y) = (3*x + y^2) * abs(sin(x) + cos(y))

p_level_surface = surface(
    x, y, g,
    title="Поверхность функции g(x, y)",
    xlabel="x",
    ylabel="y",
    zlabel="g(x, y)",
    color=:viridis,
    legend=false,
    size=(800, 600)
)
```

[58]:

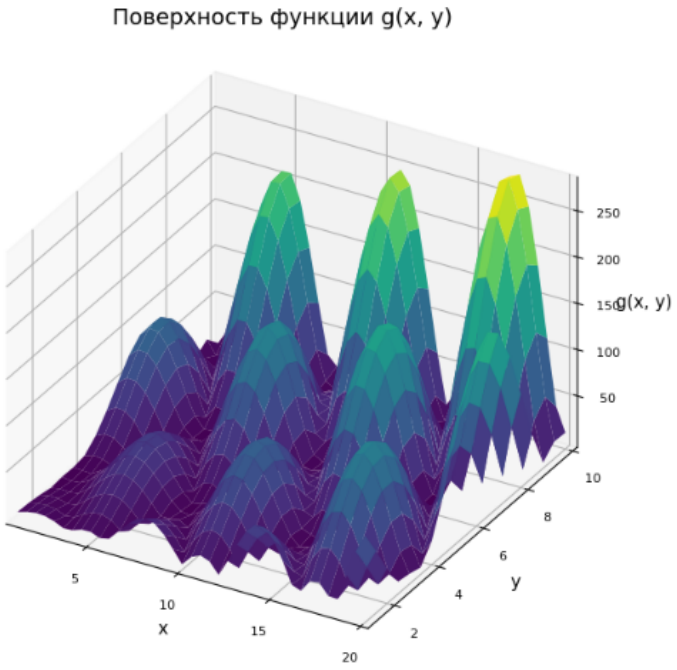
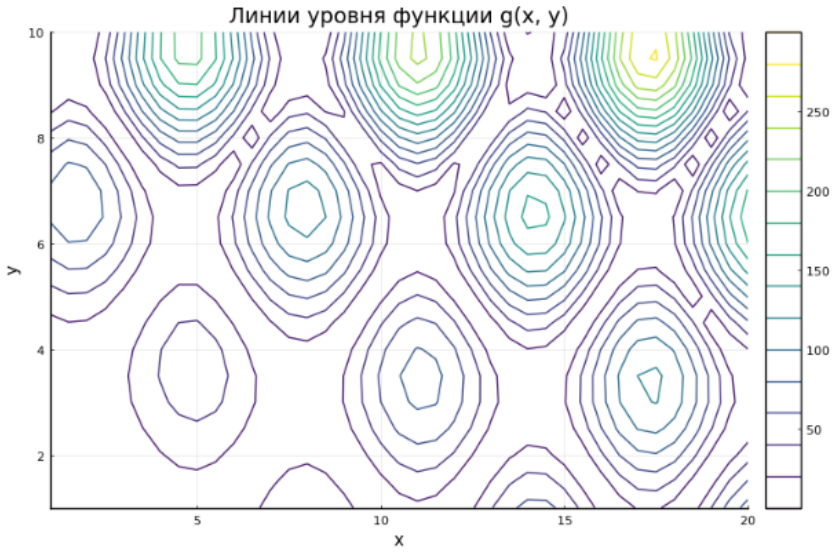


Рисунок 4.20: Поверхность функции g(x,y)

Затем с помощью `contour()` были построены линии уровня (Рисунок 4.21).

[59]:
```
p_contour = contour(
    x, y, g,
    title="Линии уровня функции g(x, y)",
    xlabel="x",
    ylabel="y",
    color=:viridis,
    size=(800, 500)
)
```

[59]:




```
sys:1: UserWarning: The following kwargs were not used by contour: 'label'
```

Рисунок 4.21: Линии уровня функции g(x,y)

При добавлении параметра `fill=true` области между уровнями были заполнены цветом (Рисунок 4.22).

[60]:
```
p_contour_fill = contour(
    x, y, g,
    fill=true,
    title="Линии уровня с заливкой",
    xlabel="x",
    ylabel="y",
    color=:viridis,
    size=(800, 500)
)
```

[60]:

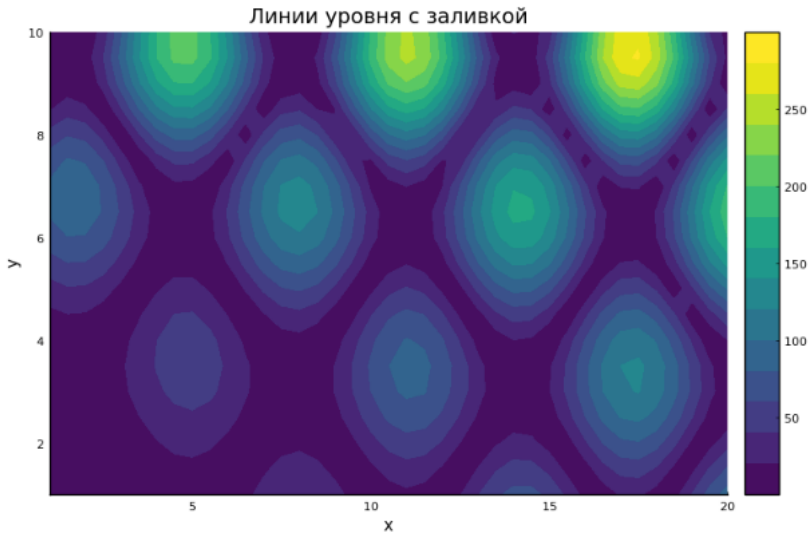


Рисунок 4.22: Линии уровня с цветной заливкой

Цветовая шкала позволяет сопоставить области рисунка со значениями функции.

4.10 Векторные поля

Для функции

$$h(x, y) = x^3 - 3x + y^2$$

были построены поверхность и линии уровня. Затем было добавлено градиентное поле:

$$\nabla h(x, y) = (3x^2 - 3,\ 2y).$$

Для отображения стрелок использовалась команда `quiver!()`. Компоненты

векторов делились на 25 для уменьшения их длины.

После ограничения области видимости получен результат, представленный на Рисунок 4.23.

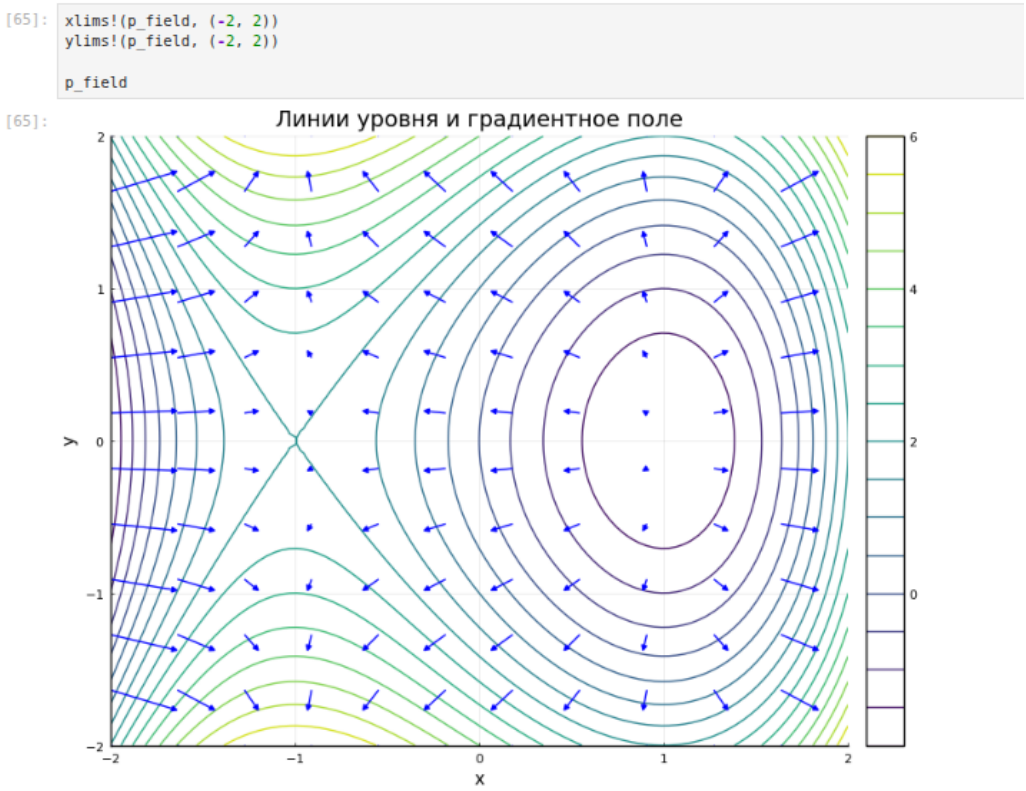


Рисунок 4.23: Линии уровня и градиентное поле

Стрелки показывают направление увеличения функции.

4.11 Анимация поверхности

Для создания анимации использовалась поверхность

$$z = \sin(x + 10\sin(\varphi)) + \cos(y).$$

Параметр φ изменялся от 0 до 2π. Сначала был построен неподвижный график (Рисунок 4.24).

5.2.11. Анимация

5.2.11.1. GIF-анимация поверхности

[66]:
```
pyplot()

X = range(-5, stop=5, length=40)
Y = range(-5, stop=5, length=40)

i = 0

p_wave = surface(
    X, Y,
    (x, y) -> sin(x + 10*sin(i)) + cos(y),
    title="Волновая поверхность",
    xlabel="x",
    ylabel="y",
    zlabel="z",
    zlims=(-2, 2),
    color=:viridis,
    legend=false,
    size=(700, 500)
)
```

[66]:

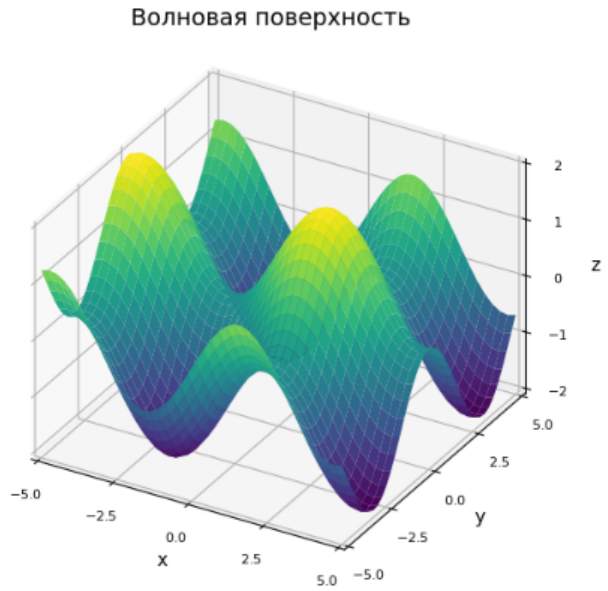


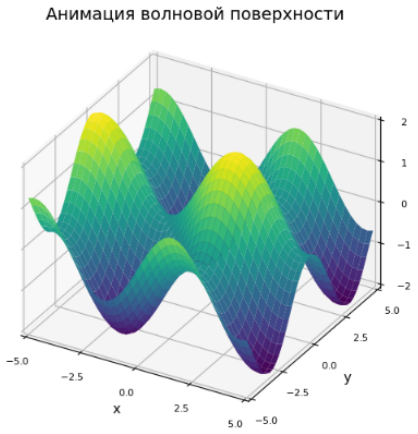
Рисунок 4.24: Исходная волновая поверхность

Затем с помощью `@animate` была сформирована последовательность из 100 кадров (Рисунок 4.25).

[67]:
```
anim_surface = @animate for phase in range(0, stop=2π, length=100)

    surface(
        X, Y,
        (x, y) -> sin(x + 10*sin(phase)) + cos(y),
        title="Анимация волновой поверхности",
        xlabel="x",
        ylabel="y",
        zlabel="z",
        zlims=(-2, 2),
        color=:viridis,
        clims=(-2, 2),
        legend=false,
        size=(700, 500)
    )

end
```



[67]:
```
Animation("/tmp/jl_VprHk7", ["000001.png", "000002.png", "000003.png", "000004.png", "000005.png", "000006.png", "000007.png", "000008.png", "000009.png", "000010.png"  …  "000091.png", "000092.png", "000093.png", "000094.png", "000095.png", "000096.png", "000097.png", "000098.png", "000099.png", "000100.png"])
```

Рисунок 4.25: Формирование кадров анимации поверхности

В представленном скриншоте показан объект анимации и один из кадров. Проверка воспроизведения GIF требует самого файла анимации.

4.12 Построение гипоциклоиды

Гипоциклоида описывает движение точки окружности, которая катится по внутренней стороне другой окружности.

Для радиусов R и r и отношения $k = R/r$ использовались формулы

$$x(t) = (R - r)\cos(t) + r\cos((k - 1)t),$$

$$y(t) = (R - r)\sin(t) - r\sin((k - 1)t).$$

Была построена большая окружность, затем добавлена часть гипоциклоиды (Рисунок 4.26).

[70]:
```
i = 51
t = θ[1:i]

xh = r_small*(k-1) .* cos.(t) .+
     r_small .* cos.((k-1) .* t)

yh = r_small*(k-1) .* sin.(t) .-
     r_small .* sin.((k-1) .* t)

plot!(p_hypo, xh, yh, color=:red, linewidth=2)
```

[70]:

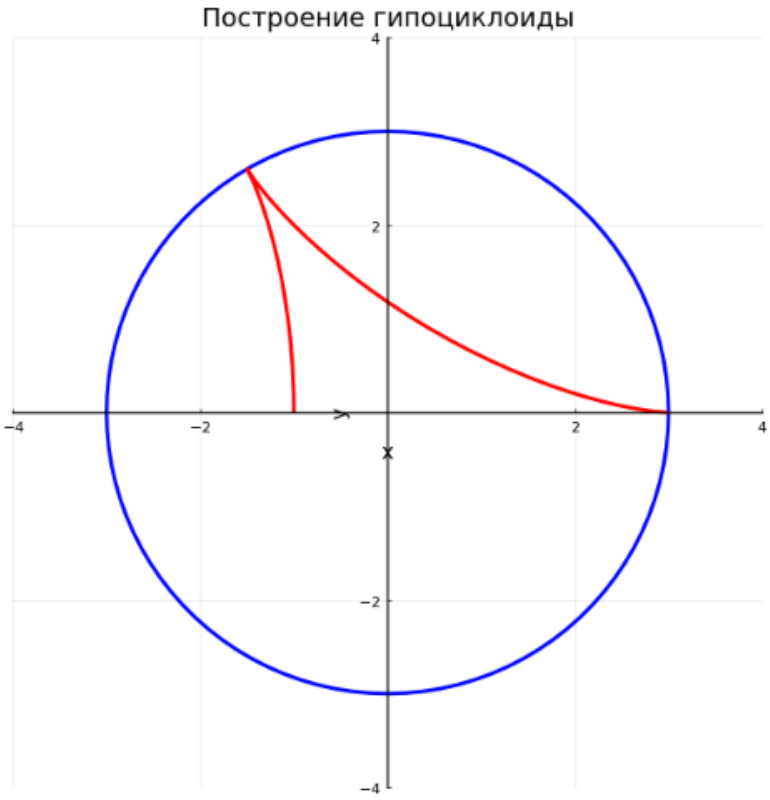


Рисунок 4.26: Частичное построение гипоциклоиды

При первоначальном построении вспомогательной окружности были допущены ошибки в вычислении её центра. Поэтому на промежуточных изображениях вспомогательная окружность и радиус отображались некорректно.

Корректное построение центра и движущейся окружности было реализовано позднее в функции `make_hypocycloid`, использованной в самостоятельном задании 10.

4.13 Погрешности измерений

Были сгенерированы выборки с различными стандартными отклонениями. Для каждой выборки вычислялось среднее значение.

Полуширина приблизительного интервала рассчитывалась по формуле

$$\Delta = 1.96\frac{\sigma}{\sqrt{n}},$$

где σ является заданным стандартным отклонением, а n размером выборки.

Вертикальные интервалы были построены с помощью `yerror` (Рисунок 4.27).

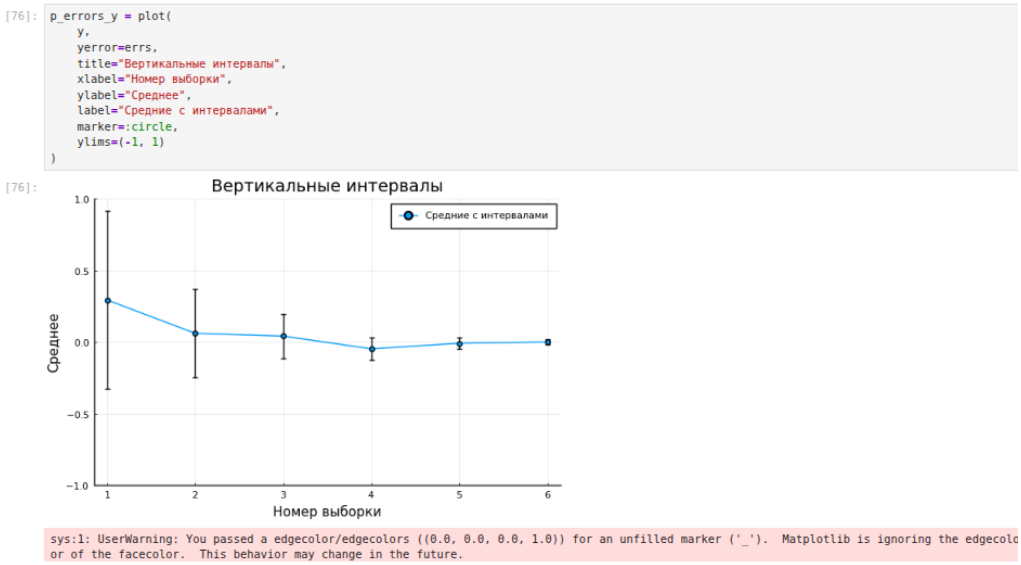
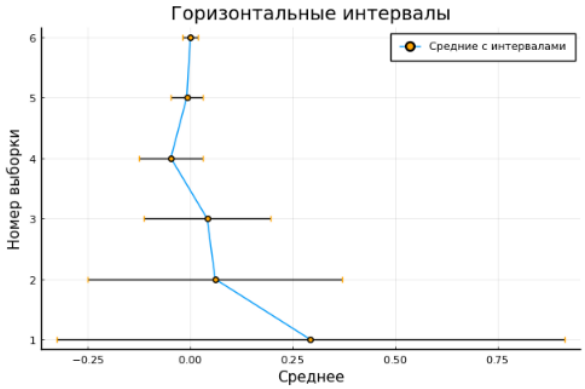


Рисунок 4.27: Вертикальные интервалы

Горизонтальные интервалы задавались параметром `xerror` (Рисунок 4.28).

```
[77]: p_errors_x = plot(
          y, 1:length(y),
          xerror=errs,
          title="Горизонтальные интервалы",
          xlabel="Среднее",
          ylabel="Номер выборки",
          label="Средние с интервалами",
          marker=:circle,
          markercolor=:orange
      )
```

[77]:




```
sys:1: UserWarning: You passed a edgecolor/edgecolors ((0.0, 0.0, 0.0, 1.0)) for an unfilled marker ('|').  Matplotlib is ignorin
or of the facecolor.  This behavior may change in the future.
```

Рисунок 4.28: Горизонтальные интервалы

Параметр `ribbon` использовался для отображения неопределённости в виде цветной полосы (Рисунок 4.29).

[78]:
```
p_ribbon = plot(
    y,
    ribbon=errs,
    fillcolor=:cyan,
    fillalpha=0.4,
    title="Полоса неопределённости",
    xlabel="Номер выборки",
    ylabel="Среднее",
    label="Средние с интервалами",
    marker=:circle
)
```

[78]:

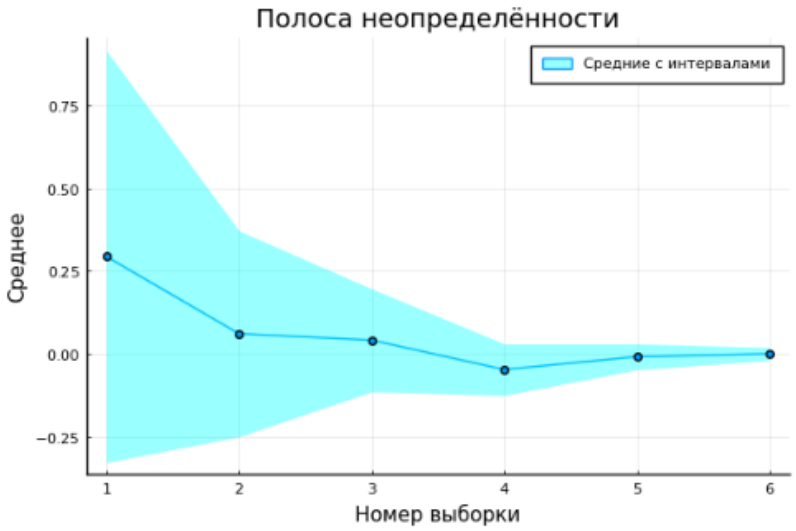


Рисунок 4.29: Полоса неопределённости

Для пяти групп наблюдений были вычислены средние значения и приближённые интервалы по обеим координатам (Рисунок 4.30).

[79]:
```
n = 10

# Пять групп по десять наблюдений
x_samples = [(rand()+1) .* randn(n) .+ 2*i for i in 1:5]
y_samples = [(rand()+1) .* randn(n) .+ i for i in 1:5]

# Полуширина приблизительного интервала для среднего
interval95(v) = 1.96 * std(v) / sqrt(length(v))

x_err = map(interval95, x_samples)
y_err = map(interval95, y_samples)

x_mean = map(mean, x_samples)
y_mean = map(mean, y_samples)
```

[79]:
```
5-element Vector{Float64}:
 0.3973754203495733
 2.2314785248124016
 3.043319295071808
 3.812021335071402
 4.937841490897019
```

[80]:
```
p_errors_xy = plot(
    x_mean, y_mean,
    xerror=x_err,
    yerror=y_err,
    title="Интервалы по двум осям",
    xlabel="Среднее x",
    ylabel="Среднее y",
    label="Средние значения",
    marker=:circle,
    markercolor=:orange
)
```

[80]:

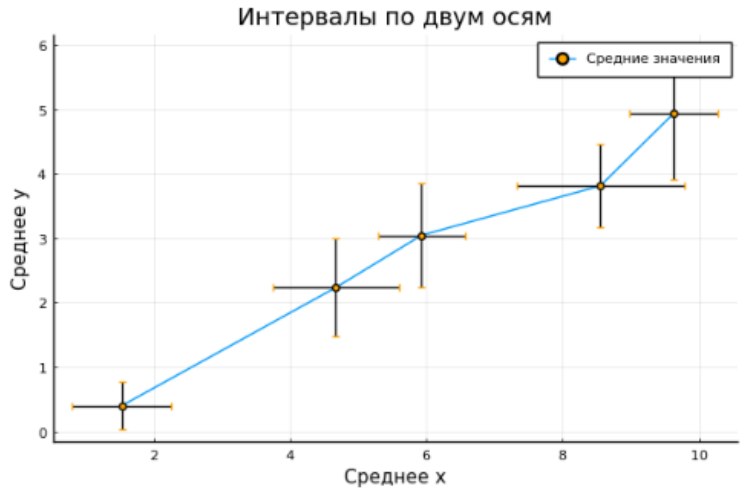


Рисунок 4.30: Интервалы по двум осям

Также был рассмотрен вариант асимметричного отображения интервалов (Рисунок 4.31). Длины в отрицательную и положительную стороны задавались отдельно.

[81]:
```
p_errors_asym = plot(
    x_mean, y_mean,
    xerror=(0.5 .* x_err, 2 .* x_err),
    yerror=(0.5 .* y_err, 2 .* y_err),
    title="Асимметричные интервалы",
    xlabel="Среднее x",
    ylabel="Среднее y",
    label="Средние значения",
    marker=:circle,
    markercolor=:orange
)
```

[81]:

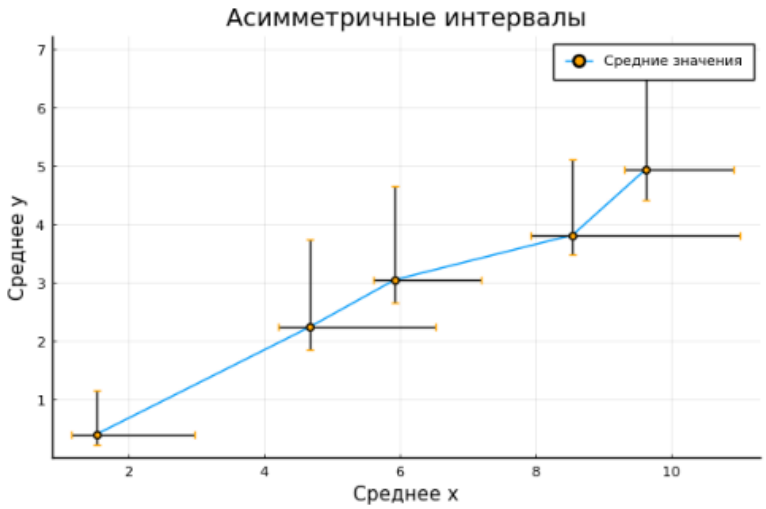


Рисунок 4.31: Асимметричные интервалы

4.14 Использование пакета Distributions

Сначала была построена гистограмма 1000 случайных целых значений возраста от 15 до 55 (Рисунок 4.32).

[84]:
```
using Distributions
```

[85]:
```
pyplot()

ages = rand(15:55, 1000)

p_ages_random = histogram(
    ages,
    title="Случайные значения возраста",
    xlabel="Возраст, лет",
    ylabel="Количество",
    label="Возраст",
    color=:blue
)
```

[85]:

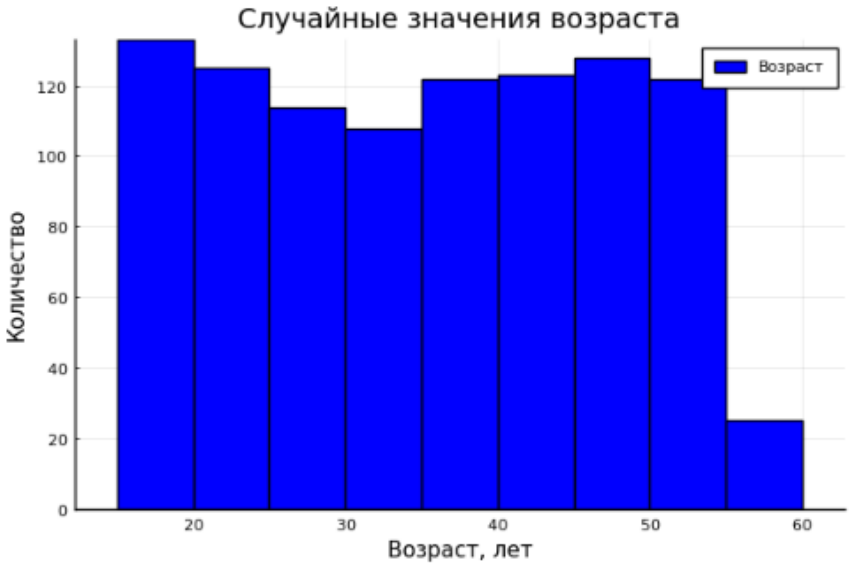


Рисунок 4.32: Гистограмма случайных возрастов

Затем использовалось нормальное распределение со средним 35 и стандартным отклонением 10 (Рисунок 4.33).

[86]:
```
d = Normal(35.0, 10.0)

ages = rand(d, 1000)

p_ages_normal = histogram(
    ages,
    title="Нормальное распределение возрастов",
    xlabel="Возраст, лет",
    ylabel="Количество",
    label="Возраст",
    color=:green
)
```

[86]:

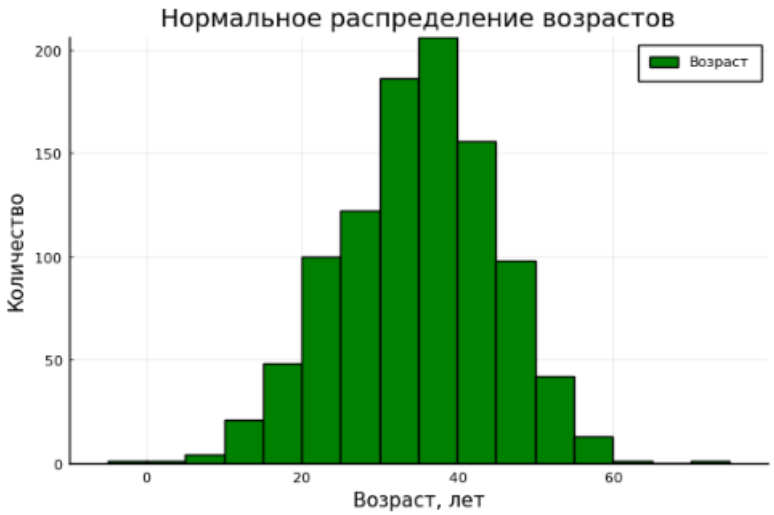


Рисунок 4.33: Гистограмма нормального распределения

Дополнительно были объединены три выборки из нормальных распределений со средними 10, 35 и 60. Полученный массив содержит 1834 значения.

Для смеси была вызвана команда построения гистограммы через Plotly (Рисунок 4.34). На представленном скриншоте виден код построения, но отсутствует изображение результата.

```
[88]: plotly()

      p_ages_mix = histogram(
          ages,
          bins=50,
          title="Смесь трёх распределений возрастов",
          xlabel="Возраст, лет",
          ylabel="Количество",
          label="Возраст",
          color=:blue
      )

      p_ages_mix
```

Рисунок 4.34: Код построения гистограммы смеси распределений

Сгенерированные возраста используются как учебные данные. Нормальное распределение не ограничено реалистичным диапазоном возрастов.

4.15 Подграфики

Для четырёх серий данных были рассмотрены различные способы расположения графиков.

При `layout=(4,1)` графики размещались друг под другом (Рисунок 4.35).

5.2.14. Подграфики

[89]:
```
pyplot()

x = range(-2, 2, length=10)
y = rand(10, 4)
```

[89]:
```
10×4 Matrix{Float64}:
 0.848376  0.434705  0.591964  0.579213
 0.581851  0.712836  0.968256  0.783836
 0.354989  0.222189  0.904805  0.217652
 0.688229  0.61071   0.323353  0.247915
 0.906437  0.134705  0.400965  0.777951
 0.88584   0.472263  0.235986  0.452115
 0.29019   0.597162  0.89051   0.639891
 0.100826  0.824924  0.26888   0.268276
 0.494475  0.425257  0.60993   0.416625
 0.384684  0.678511  0.998616  0.465771
```

[90]:
```
plot(
    x, y,
    layout=(4, 1),
    title=["Серия 1" "Серия 2" "Серия 3" "Серия 4"],
    xlabel="x",
    ylabel="y",
    legend=false,
    size=(800, 800)
)
```

[90]:

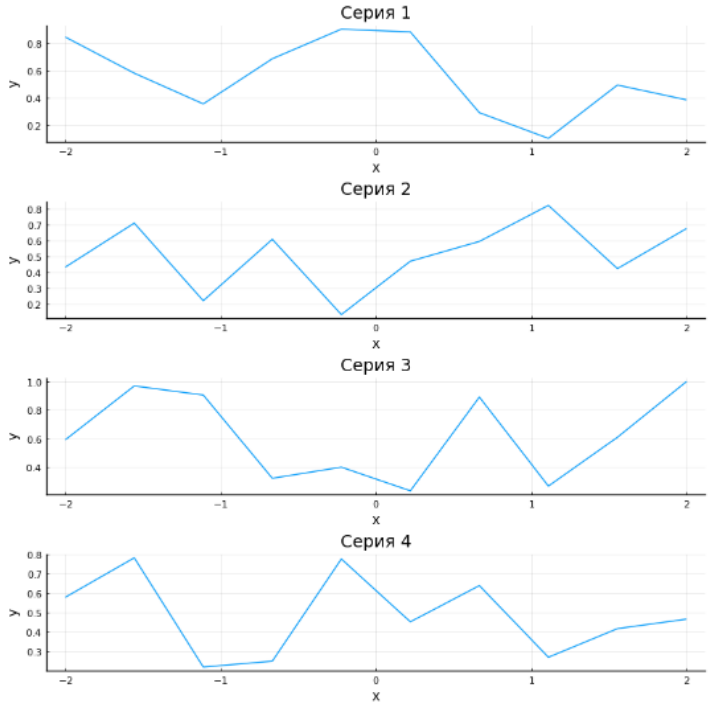


Рисунок 4.35: Четыре графика в одном столбце

При `layout=4` автоматически формировалась сетка 2 × 2 (Рисунок 4.36).

```
[91]: plot(
          x, y,
          layout=4,
          title=["Серия 1" "Серия 2" "Серия 3" "Серия 4"],
          xlabel="x",
          ylabel="y",
          legend=false,
          size=(800, 600)
      )
```

[91]:

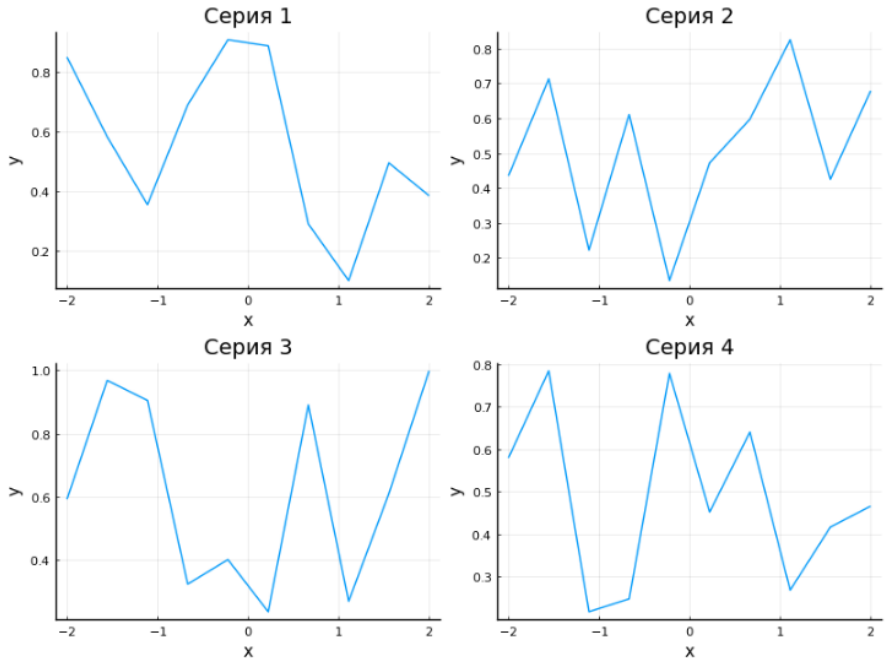


Рисунок 4.36: Автоматическое расположение четырёх графиков

С помощью `grid` были заданы разные доли высоты подграфиков. Использовались значения 0.15, 0.25, 0.4 и 0.2, сумма которых равна единице.

Затем в одном окне были объединены линейный график, точечный график, две серии линий и гистограммы (Рисунок 4.37).

```
[93]: p1 = plot(
          x, y,
          title="Линии",
          xlabel="x", ylabel="y"
      )

      p2 = scatter(
          x, y,
          title="Точки",
          xlabel="x", ylabel="y"
      )

      p3 = plot(
          x, y[:, 1:2],
          title="Две серии",
          xlabel="x", ylabel="y",
          linewidth=2
      )

      p4 = histogram(
          y,
          title="Гистограммы",
          xlabel="Значение", ylabel="Количество",
          alpha=0.5
      )

      p_combined = plot(
          p1, p2, p3, p4,
          layout=(2, 2),
          legend=false,
          size=(900, 700),
          background_color=:ivory
      )
```

[93]:

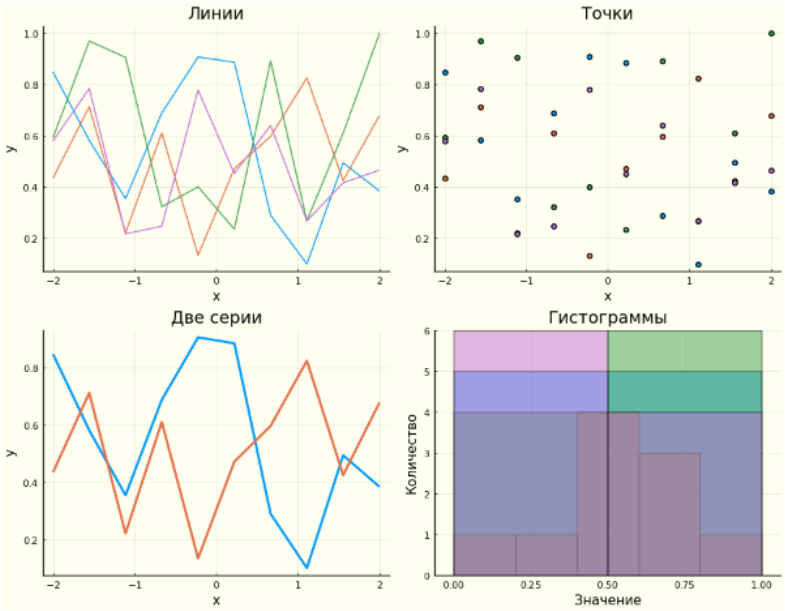


Рисунок 4.37: Объединение графиков разных типов

В отдельном примере с шестью типами представления данных фактически заполнился только первый подграфик. Этот пример требует корректировки распределения данных по подграфикам.

Для сложного макета использовался макрос `@layout`. Были размещены 11 подграфиков различного размера (Рисунок 4.38).

[95]:
```
layout_complex = @layout [
    a{0.3w} [grid(3, 3)
             b{0.2h}]
]

p_complex = plot(
    rand(10, 11),
    layout=layout_complex,
    legend=false,
    seriestype=[:bar :scatter :path],
    title=["($i)" for j in 1:1, i in 1:11],
    titleloc=:right,
    titlefont=font(8),
    size=(1000, 700)
)
```

[95]:

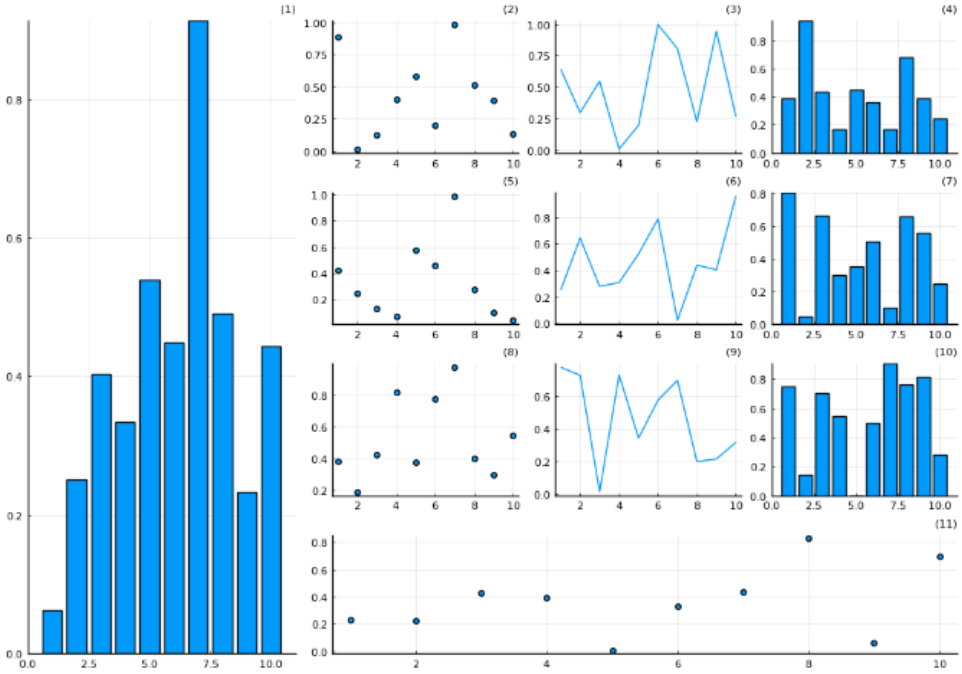


Рисунок 4.38: Сложный макет из одиннадцати подграфиков

5 Самостоятельные задания

5.1 Задание 1. Типы графиков функции sin(x)

На отрезке $[0, 2\pi]$ были вычислены значения функции $\sin(x)$.

В одном окне построены:

1. Линейный график.
2. Точечный график.
3. Линия с маркерами.
4. Столбчатый график.
5. Стержневой график.
6. Ступенчатый график.
7. График с заливкой.
8. Гистограмма значений функции.

Результат представлен на Рисунок 5.1.

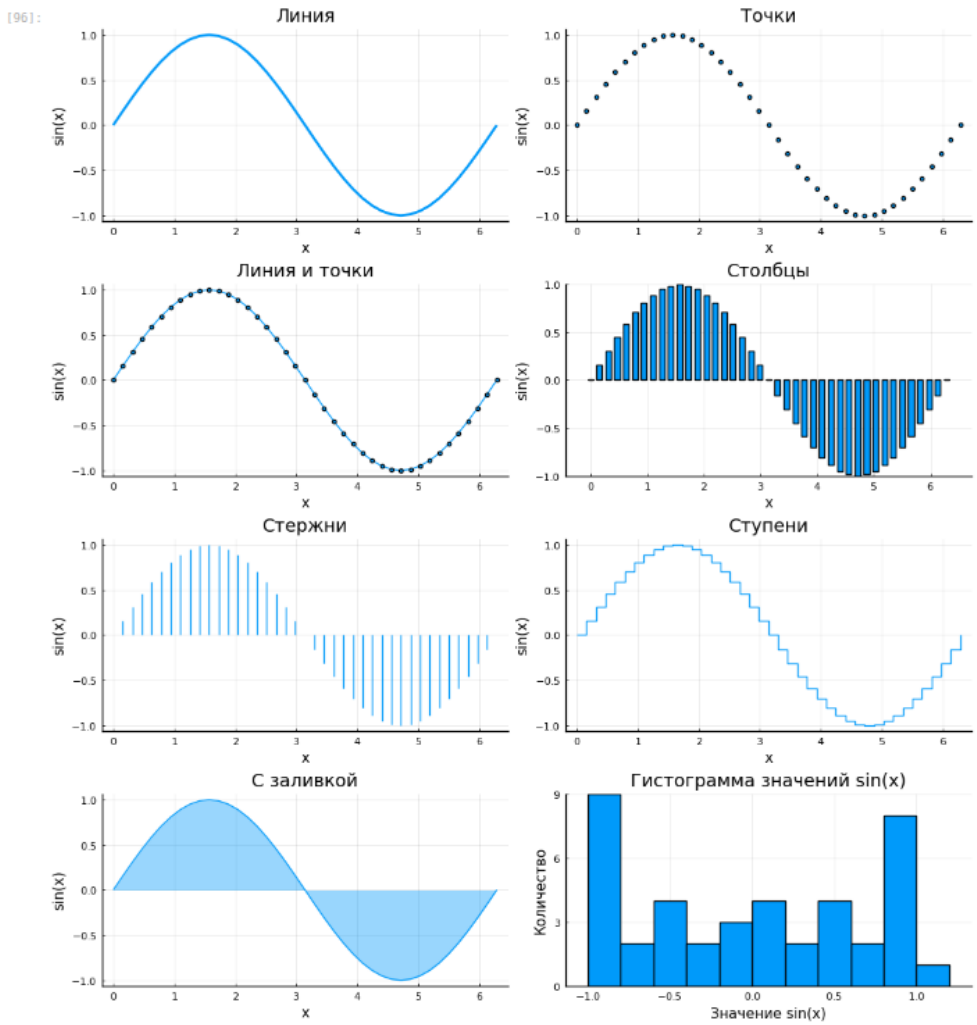


Рисунок 5.1: Различные способы представления значений sin(x)

Первые семь подграфиков показывают зависимость от аргумента. Гистограмма показывает распределение вычисленных значений синуса по интервалам.

5.2 Задание 2. Стили линий

Для функции $\sin(x)$ были заданы сплошной, штриховой, точечный и штрих-пунктирный стили.

Также была предпринята попытка использования `:dashdotdot`. Графический механизм PyPlot сообщил, что данный стиль не поддерживается. Поэтому пятый

график нельзя считать отдельным успешно отображённым стилем (Рисунок 5.2).

Задание 2. Стили линий графика sin(x)

[97]:
```
x = range(0, stop=2π, length=300)
y = sin.(x)

styles = [:solid, :dash, :dot, :dashdot, :dashdotdot]

style_names = [
    "Сплошная",
    "Штриховая",
    "Точечная",
    "Штрихпунктирная",
    "Штрих и две точки"
]

plots_styles = []

for j in eachindex(styles)

    p = plot(
        x, y,
        linestyle=styles[j],
        linewidth=2,
        title=style_names[j],
        xlabel="x",
        ylabel="sin(x)",
        label="sin(x)",
        color=:blue,
        ylims=(-1.2, 1.2)
    )

    push!(plots_styles, p)

end

p_sin_styles = plot(
    plots_styles...,
    layout=(3, 2),
    legend=false,
    size=(1000, 800)
)
```

```
[ Warning: linestyle dashdotdot is unsupported with Plots.PyPlotBackend(). Choose from: [:auto, :dash, :dashdot, :dot, :solid]
@ Plots ~/.julia/packages/Plots/h49MV/src/args.jl:1579
[ Warning: Unknown linestyle dashdotdot
@ Plots ~/.julia/packages/Plots/h49MV/src/backends/deprecated/pyplot.jl:106
[ Warning: Unknown linestyle dashdotdot
@ Plots ~/.julia/packages/Plots/h49MV/src/backends/deprecated/pyplot.jl:106
[ Warning: Unknown linestyle dashdotdot
@ Plots ~/.julia/packages/Plots/h49MV/src/backends/deprecated/pyplot.jl:106
[ Warning: Unknown linestyle dashdotdot
@ Plots ~/.julia/packages/Plots/h49MV/src/backends/deprecated/pyplot.jl:106
```

[97]:

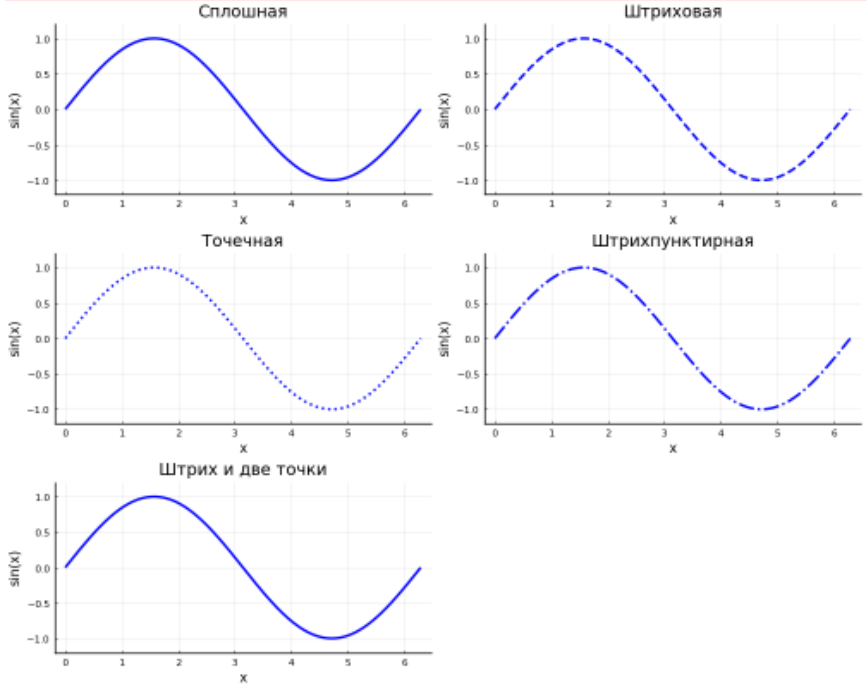


Рисунок 5.2: Сравнение стилей линий и предупреждение о неподдерживаемом стиле

5.3 Задание 3. Оформление графика

Была построена функция

$$y = \pi x^2 \ln(x), \qquad x > 0.$$

Для вычислений использовался диапазон от 0.1 до 3. Были настроены красный цвет линии, зелёный цвет рамки, шрифты, отметки на осях и поля рисунка (Рисунок 5.3).

[98]:
```
using Plots.PlotMeasures

x = 0.1:0.05:3
y = π .* x.^2 .* log.(x)

p_format = plot(
    x, y,
    title="График y = πx²ln(x)",
    xlabel="Аргумент x",
    ylabel="Значение y",
    label="πx²ln(x)",

    linecolor=:red,
    linewidth=2,

    framestyle=:box,
    foreground_color_border=:green,

    titlefont=font(14, "DejaVu Serif"),
    guidefont=font(12, "DejaVu Serif"),
    tickfont=font(10, "DejaVu Serif"),

    xticks=0:0.5:3,
    yticks=0:5:35,

    left_margin=10mm,
    bottom_margin=10mm,
    right_margin=5mm,
    top_margin=5mm,

    size=(800, 500),
    grid=true
)
```

```
WARNING: using PlotMeasures.h in module Main conflicts with an existing identifier.
WARNING: using PlotMeasures.px in module Main conflicts with an existing identifier.
```

[98]:

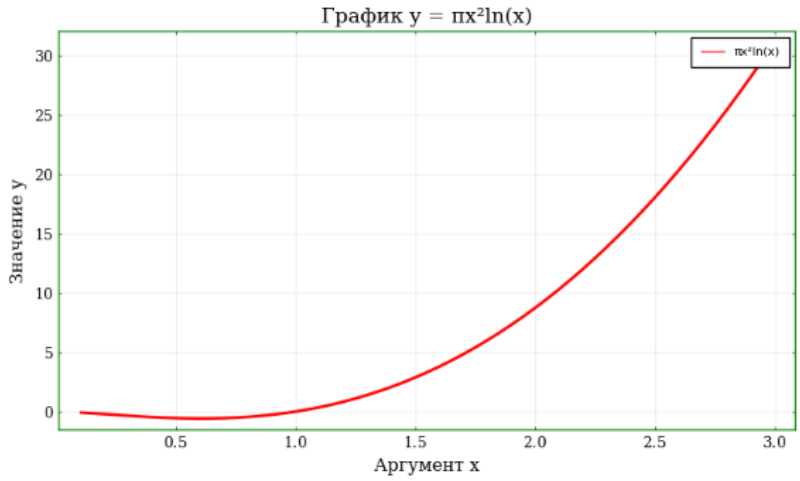


Рисунок 5.3: Оформление графика функции с логарифмом

Увеличение полей позволило разместить подписи осей в пределах изображения.

5.4 Задание 4. Четыре способа изображения функции

Для вектора

$$x = (-2, -1, 0, 1, 2)$$

были вычислены значения функции

$$y = x^3 - 3x.$$

Получены значения

$$y = (-2, 2, 0, -2, 2).$$

В четырёх подокнах были построены точки, ломаная, ломаная с маркерами и плавная кривая (Рисунок 5.4).

[100]:
```
p1 = scatter(
    x, y,
    title="Точки",
    color=:blue,
    markersize=6
)

p2 = plot(
    x, y,
    title="Линии",
    color=:blue,
    linewidth=2
)

p3 = plot(
    x, y,
    title="Линии и точки",
    color=:blue,
    linewidth=2,
    marker=:circle,
    markersize=6
)

p4 = plot(
    x_curve, y_curve,
    title="Плавная кривая",
    color=:red,
    linewidth=2
)

p_four = plot(
    p1, p2, p3, p4,
    layout=(2, 2),
    xlabel="x",
    ylabel="y",
    legend=false,
    size=(900, 650)
)
```

[100]:

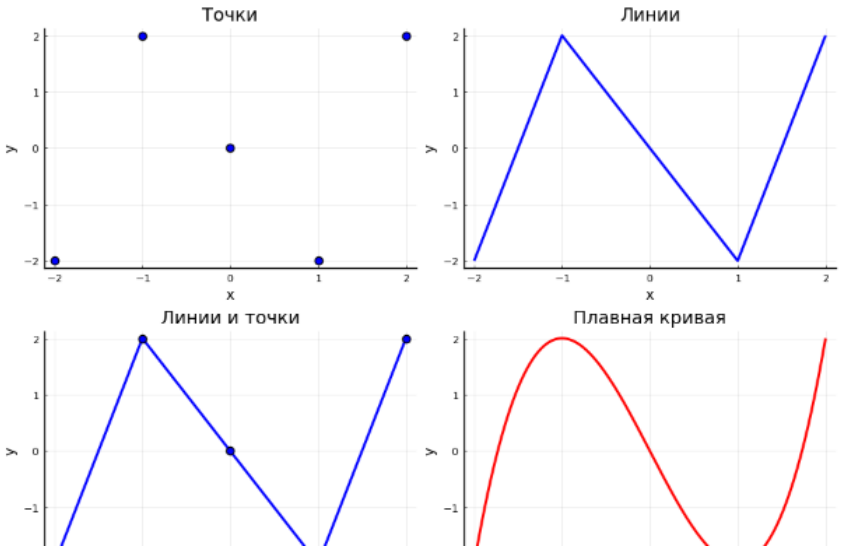


Рисунок 5.4: Четыре способа изображения функции

Для плавной кривой использовались 300 значений аргумента. При построении ломаной использовались только пять заданных точек.

В коде было предусмотрено сохранение результата в файл `figure_nazarmamadov.png`.

5.5 Задание 5. Функции с различными масштабами

На отрезке от 3 до 6 с шагом 0.1 были построены функции

$$y_1 = \pi x, \qquad y_2 = e^x \cos(x).$$

Сначала использовалась общая вертикальная шкала (Рисунок 5.5).

[101]:
```
pyplot()

x = collect(3:0.1:6)

y1 = π .* x
y2 = exp.(x) .* cos.(x)

p_common = plot(
    x, y1,
    title="Две функции с общей шкалой",
    xlabel="x",
    ylabel="y",
    label="y1 = πx",
    color=:blue,
    linewidth=2,
    grid=true,
    legend=:topleft
)

plot!(
    p_common,
    x, y2,
    label="y2 = exp(x)cos(x)",
    color=:red,
    linewidth=2
)
```

[101]:

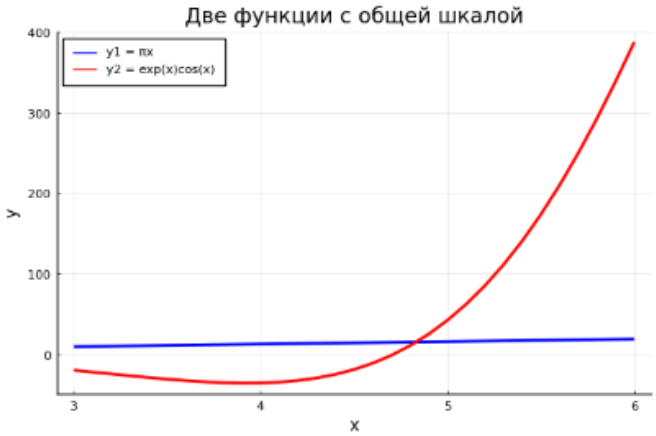


Рисунок 5.5: Две функции с общей вертикальной шкалой

Недостатком такого представления является слабая заметность изменения первой функции на фоне значительно большего диапазона второй.

Затем для второй функции была добавлена отдельная правая ось (Рисунок 5.6).

[102]:
```
p_dual = plot(
    x, y1,
    title="Две функции с разными шкалами",
    xlabel="x",
    ylabel="y1 = πx",
    label="y1 = πx",
    color=:blue,
    linewidth=2,
    legend=:topleft,
    grid=true,
    size=(850, 550)
)

plot!(
    twinx(),
    x, y2,
    ylabel="y2 = exp(x)cos(x)",
    label="y2 = exp(x)cos(x)",
    color=:red,
    linewidth=2,
    legend=:bottomright,
    grid=false
)
```

[102]:

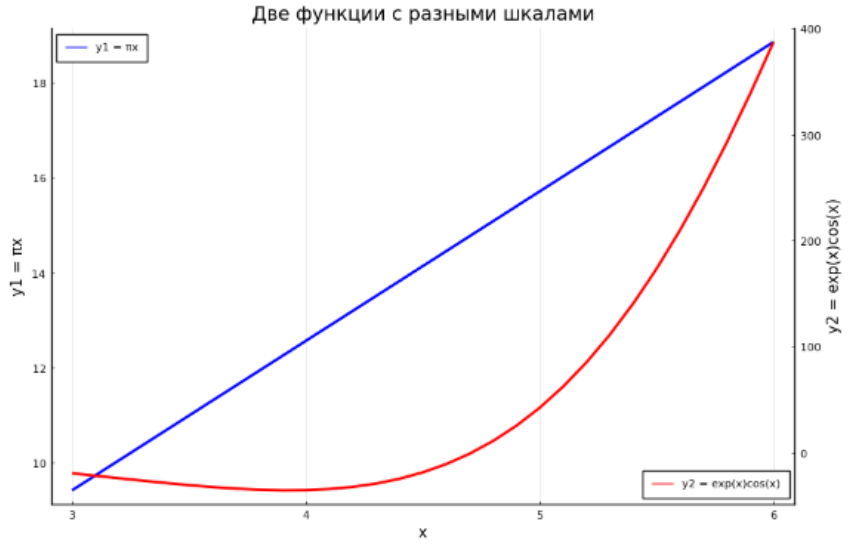


Рисунок 5.6: Две функции с отдельными вертикальными шкалами

Такое представление позволяет рассмотреть изменение обеих функций. При этом значения каждой линии необходимо читать по соответствующей шкале.

5.6 Задание 6. Данные с погрешностью измерения

Был составлен учебный набор данных о нагревании воды. Значения температуры увеличивались от 20 до 46 ℃ за пять минут.

Для каждой точки задавалась погрешность ±1 ℃ (Рисунок 5.7).

[103]:
```
time = [0, 1, 2, 3, 4, 5]
temperature = [20.0, 24.5, 29.0, 35.0, 40.5, 46.0]
errors = fill(1.0, length(temperature))

p_experiment = plot(
    time, temperature,
    yerror=errors,
    title="Нагревание воды: учебные данные",
    xlabel="Время, мин",
    ylabel="Температура, °C",
    label="Температура ±1 °C",
    color=:red,
    linewidth=2,
    marker=:circle,
    markersize=5,
    grid=true,
    legend=:topleft,
    size=(800, 500)
)
```

[103]:

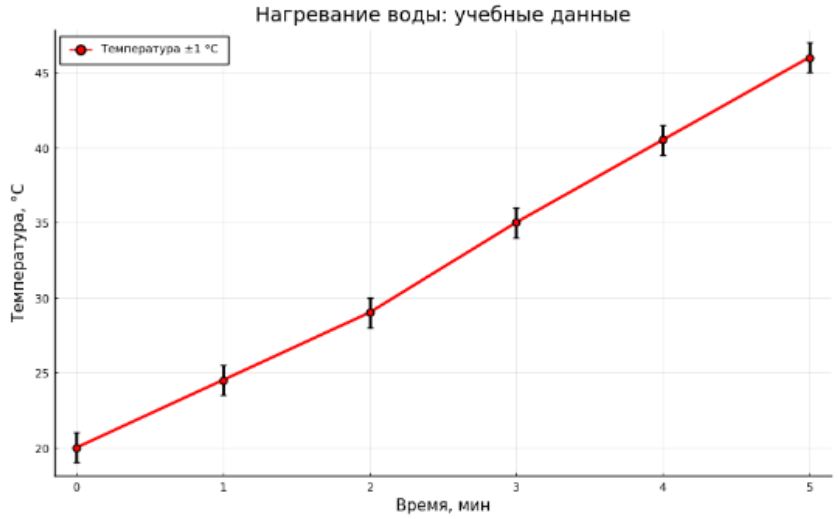


Рисунок 5.7: Учебные данные о нагревании воды

Данные являются придуманным примером и не представляют результаты реального физического эксперимента.

5.7 Задание 7. Случайные точки на плоскости

Были созданы два массива по 100 случайных чисел из диапазона от 0 до 1.

Для точечного графика заданы название, подписи осей, легенда, цвет и прозрачность маркеров (Рисунок 5.8).

[104]:
```
pyplot()

x_random = rand(100)
y_random = rand(100)

p_random2d = scatter(
    x_random, y_random,
    title="Случайные точки на плоскости",
    xlabel="Координата x",
    ylabel="Координата y",
    label="100 случайных точек",
    color=:blue,
    markersize=4,
    alpha=0.7,
    legend=:outertopright,
    size=(850, 550)
)
```

[104]:



Рисунок 5.8: Сто случайных точек на плоскости

5.8 Задание 8. Случайные точки в пространстве

Были созданы три массива по 100 случайных значений. Каждая точка задавалась тремя координатами.

Получен трёхмерный точечный график с подписями осей и легендой (Рисунок 5.9).

```
[105]: x_random3d = rand(100)
       y_random3d = rand(100)
       z_random3d = rand(100)

       p_random3d = scatter(
           x_random3d, y_random3d, z_random3d,
           title="Случайные точки в пространстве",
           xlabel="Координата x",
           ylabel="Координата y",
           zlabel="Координата z",
           label="100 случайных точек",
           color=:green,
           markersize=4,
           alpha=0.7,
           legend=:outertopright,
           size=(900, 600)
       )
```

[105]:



Рисунок 5.9: Сто случайных точек в пространстве

5.9 Задание 9. Анимация синусоиды

Для функции $\sin(x)$ на отрезке $[0, 2\pi]$ были подготовлены 120 точек.

На каждом кадре отображались значения от начала массива до текущего индекса. Красный маркер обозначал конец строящейся кривой.

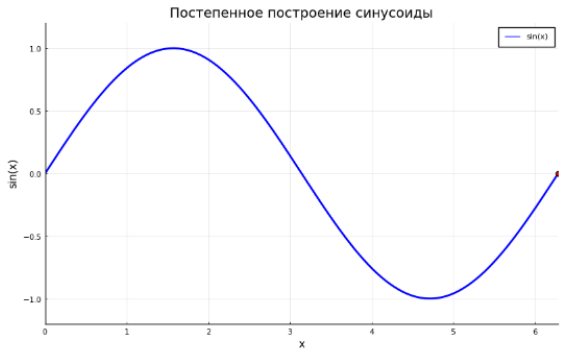
Формирование кадров представлено на Рисунок 5.10.

```
[107]: anim_sin = @animate for j in 1:length(x_sin)

           p = plot(
               x_sin[1:j],
               y_sin[1:j],
               title="Постепенное построение синусоиды",
               xlabel="x",
               ylabel="sin(x)",
               label="sin(x)",
               color=:blue,
               linewidth=2,
               xlims=(0, 2π),
               ylims=(-1.2, 1.2),
               legend=:topright,
               grid=true,
               size=(800, 500)
           )

           scatter!(
               p,
               [x_sin[j]],
               [y_sin[j]],
               color=:red,
               markersize=5,
               label=""
           )

       end
```




```
[107]: Animation("/tmp/jl_cuAXtM", ["000001.png", "000002.png", "000003.png", "000004.png", "000005.png", "000006.png", "000007.png", "000008.png", "000009.png", "000010.png"  …  "000111.png", "00011
       2.png", "000113.png", "000114.png", "000115.png", "000116.png", "000117.png", "000118.png", "000119.png", "000120.png"])
```

Рисунок 5.10: Создание анимации постепенного построения синусоиды

Пределы осей оставались постоянными на всех кадрах. В коде было предусмотрено сохранение анимации в `task09_sine.gif`.

5.10 Задание 10. Анимированные гипоциклоиды

Была создана функция `make_hypocycloid`, принимающая отношение радиусов и имя выходного файла.

Использовались два целых значения $k = 3$ и $k = 4$, а также два нецелых рациональных значения $k = 5/2$ и $k = 7/2$.

При $k = 3$ получена гипоциклоида с тремя вершинами (Рисунок 5.11).

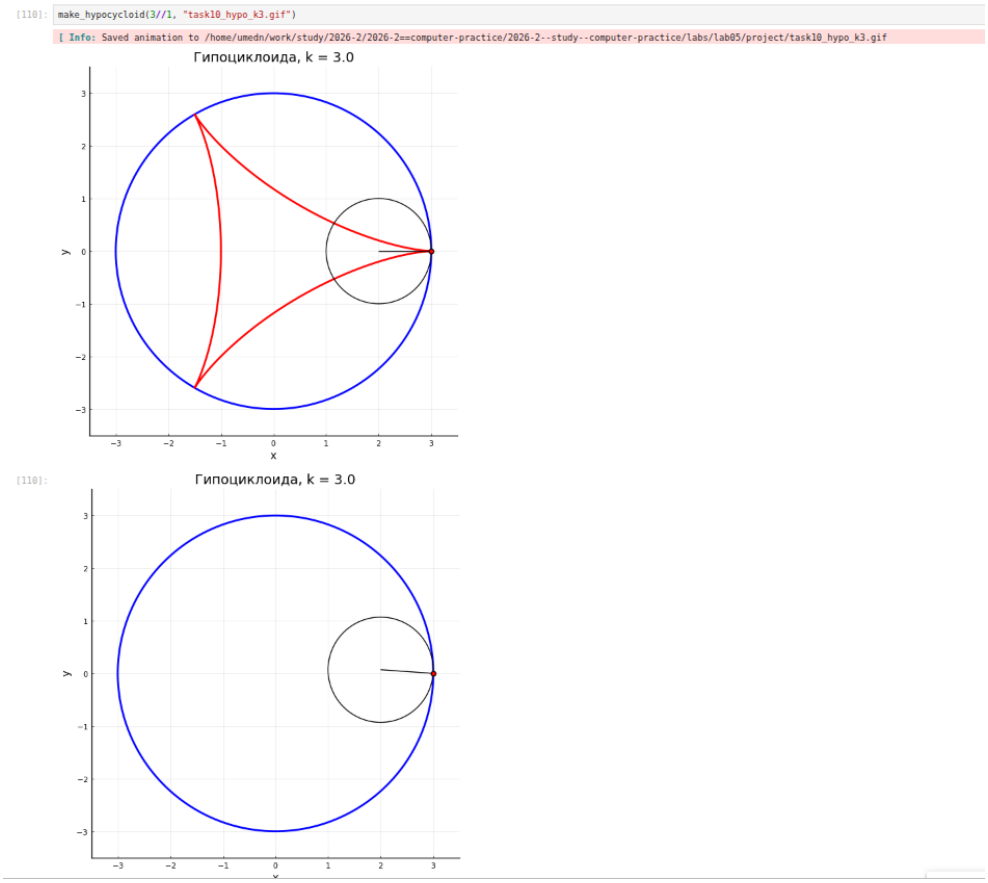


Рисунок 5.11: Гипоциклоида при k = 3

При $k = 4$ получена гипоциклоида с четырьмя вершинами (Рисунок 5.12).

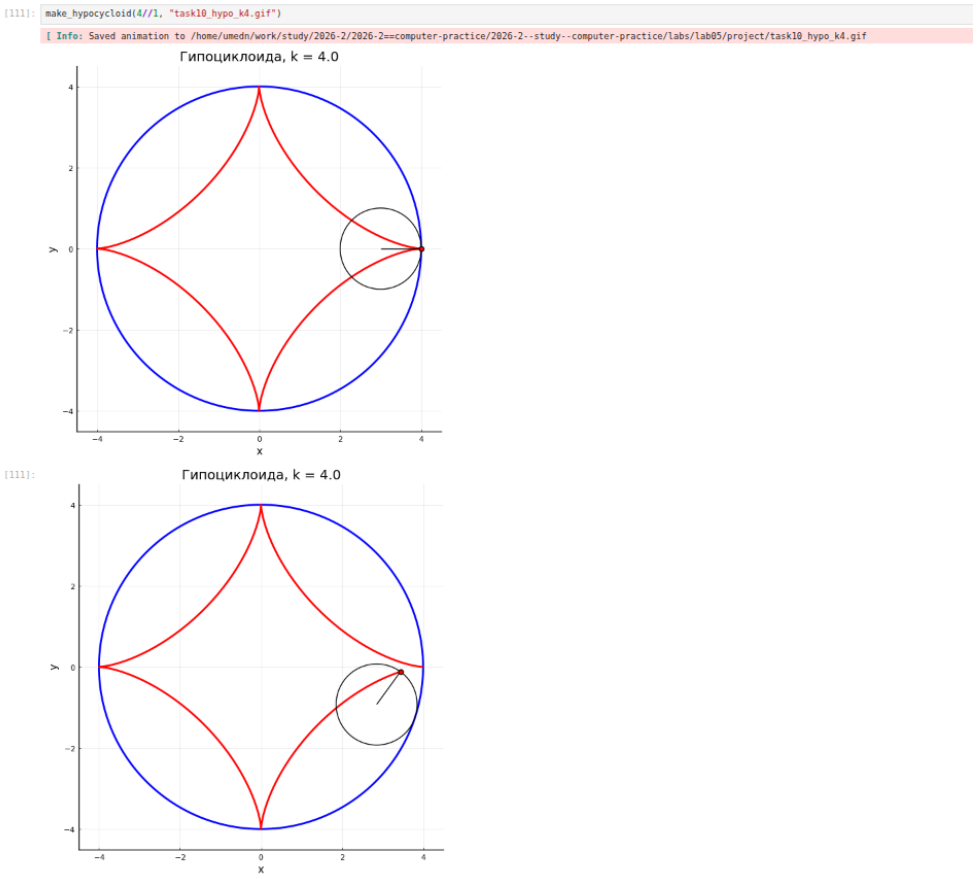


Рисунок 5.12: Гипоциклоида при k = 4

При $k = 5/2$ получена замкнутая кривая с самопересечениями (Рисунок 5.13).

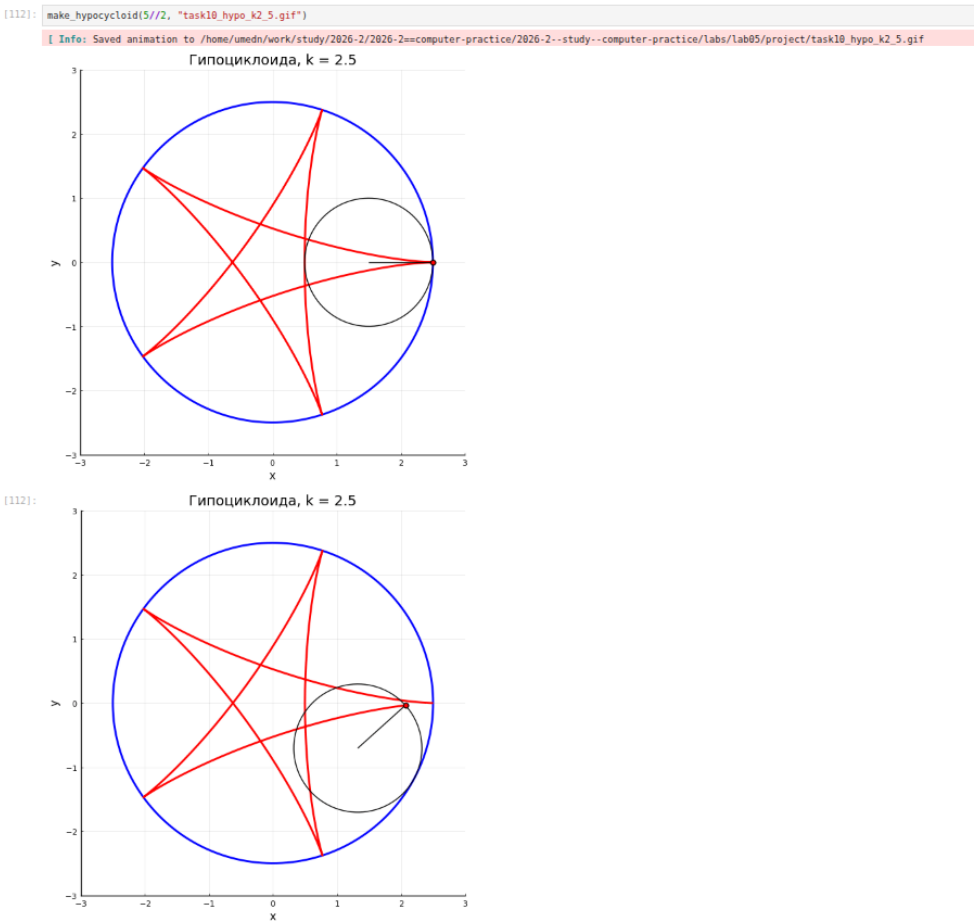


Рисунок 5.13: Гипоциклоида при k = 5/2

Аналогичное построение выполнено при $k = 7/2$ (Рисунок 5.14).

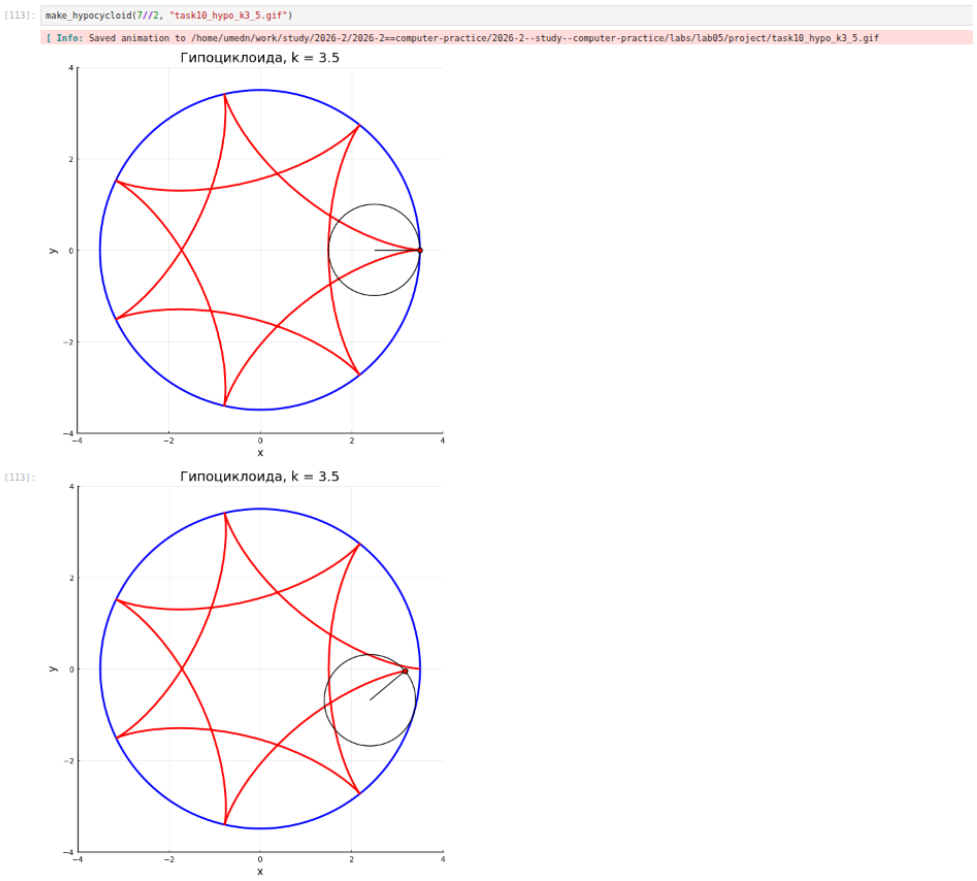


Рисунок 5.14: Гипоциклоида при k = 7/2

Для рационального отношения $k = p/q$, записанного в несократимом виде, параметр изменялся от 0 до $2\pi q$. Это обеспечивает замыкание рассматриваемых кривых.

На скриншотах присутствуют сообщения о сохранении четырёх GIF-файлов.

5.11 Задание 11. Анимированные эпициклоиды

Эпициклоида описывает движение точки окружности, катящейся по внешней стороне неподвижной окружности.

Использовались формулы

$$x(t) = (R + r)\cos(t) - r\cos((k + 1)t),$$

$$y(t) = (R + r)\sin(t) - r\sin((k + 1)t), \qquad k = \frac{R}{r}.$$

Была создана функция `make_epicycloid`. Для размещения всей кривой область отображения учитывала сумму $R + 2r$.

На Рисунок 5.15 показан результат при $k = 3$.

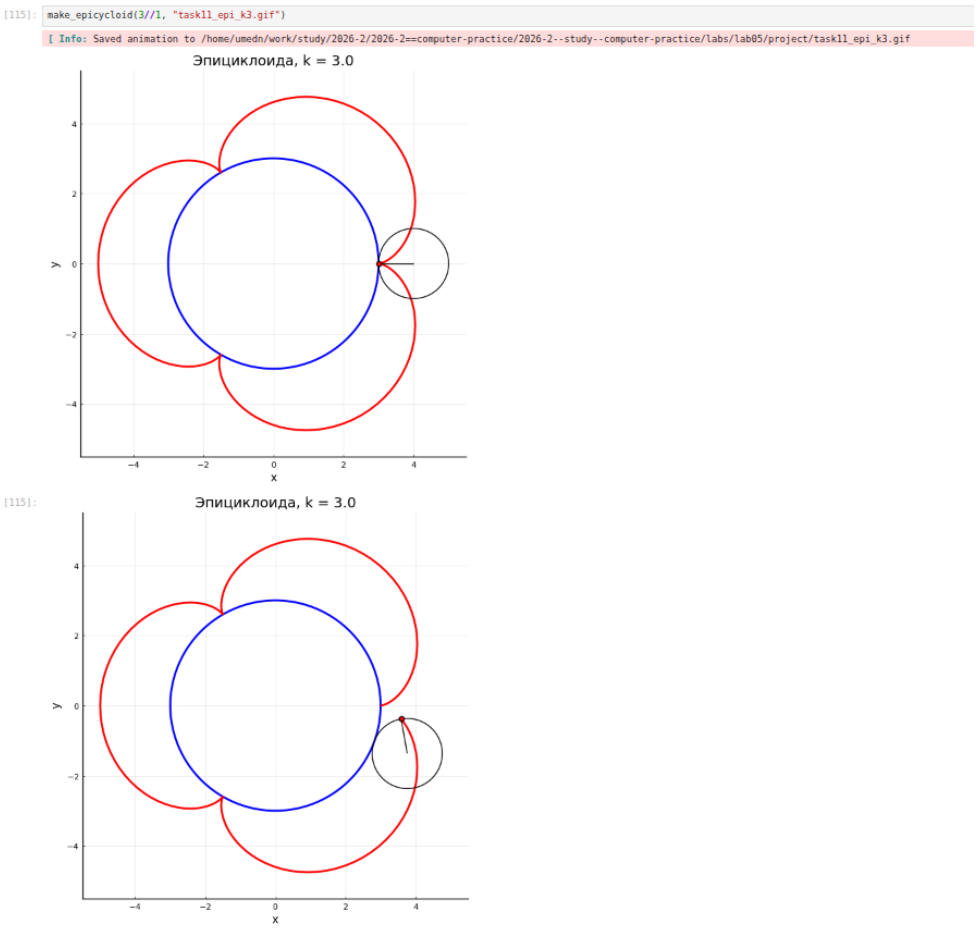


Рисунок 5.15: Эпициклоида при k = 3

На Рисунок 5.16 показан результат при $k = 4$.

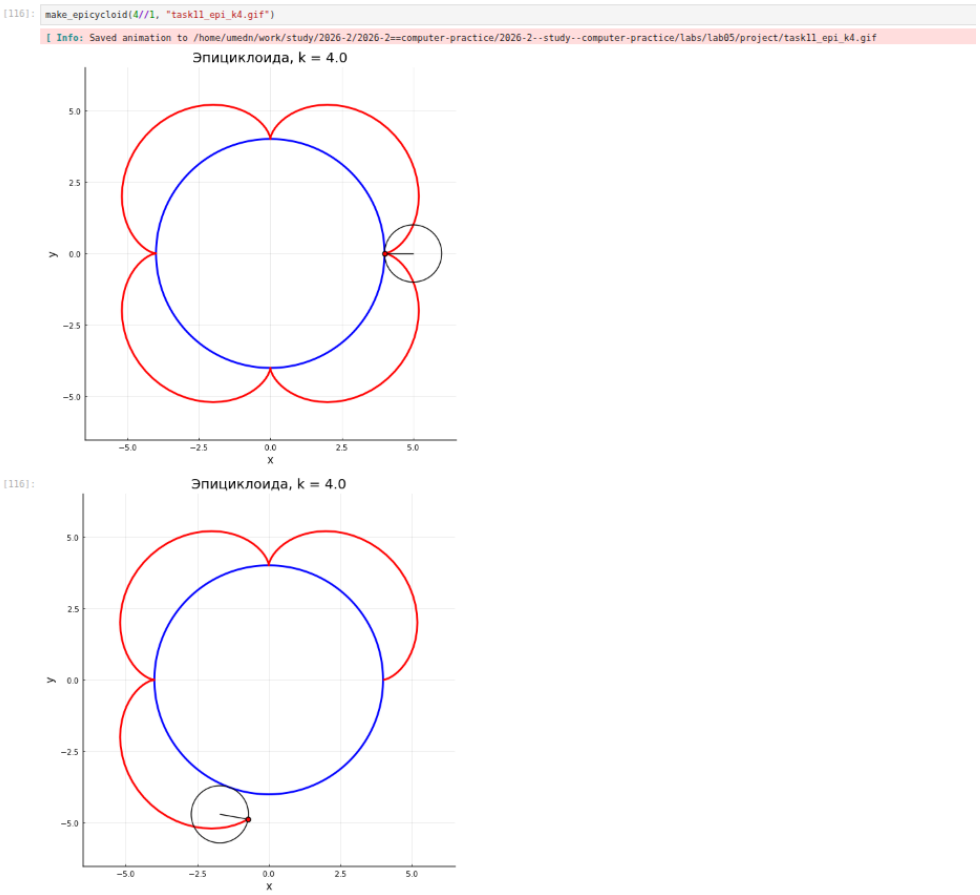


Рисунок 5.16: Эпициклоида при k = 4

На скриншотах подтверждено сохранение анимаций для двух целых значений отношения радиусов.

Функция допускает использование рациональных значений. Однако результаты её вызовов при $k = 5/2$ и $k = 7/2$ в представленных материалах отсутствуют. Для полного подтверждения выполнения задания необходимо добавить эти два результата.

6 Результаты

В ходе работы получены следующие результаты:

1. Построены графики функции с использованием различных средств визуализации.
2. Рассмотрено оформление линий, координатных осей, легенд и фона.
3. Выполнено сравнение синуса с его приближением рядом Тейлора.
4. Построены двумерные и трёхмерные точечные графики.
5. Выполнено приближение случайных данных полиномом пятой степени.
6. Построены графики с двумя вертикальными шкалами.
7. Рассмотрены полярные и параметрические графики.
8. Построены поверхности, линии уровня и градиентное поле.
9. Рассмотрены способы отображения симметричных и асимметричных интервалов.
10. Построены гистограммы случайных данных.
11. Созданы простые и сложные макеты подграфиков.
12. Сформированы последовательности кадров для анимаций.
13. Подтверждено сохранение четырёх анимаций гипоциклоид и двух анимаций эпициклоид.

7 Обсуждение результатов

Результаты показали, что выбор типа графика зависит от характера данных и цели исследования.

Линейные графики удобны для отображения изменения функции, точечные для отдельных наблюдений, а гистограммы для исследования распределения значений.

При сравнении функций с различными диапазонами значений общая шкала может скрывать изменение одной из них. Использование двух осей ординат улучшает видимость, но требует внимательного чтения подписей шкал.

Уменьшение шага сетки делает поверхность более гладкой, одновременно увеличивая количество вычисляемых точек.

Графические механизмы имеют различия в поддержке параметров и способах отображения. В работе это проявилось при выводе Plotly в JupyterLab и использовании стиля `:dashdotdot` в PyPlot.

При построении гипоциклоид выявлена необходимость корректного вычисления центра движущейся окружности. В итоговой функции центр задавался координатами

$$c_x = (R - r)\cos(t), \qquad c_y = (R - r)\sin(t).$$

Для эпициклоиды вместо $R - r$ использовалось $R + r$.

Не все результаты можно подтвердить неподвижными скриншотами. Скриншот показывает отдельный кадр, а для проверки движения необходимо открыть

соответствующий GIF-файл.

Отдельной доработки требуют пример с шестью типами подграфиков, визуальное подтверждение гистограммы смеси распределений и результаты эпициклоид для двух нецелых рациональных значений.

8 Заключение

В ходе лабораторной работы освоены основные команды Julia для построения и оформления графиков.

Получены навыки работы с линейными и точечными графиками, гистограммами, поверхностями, линиями уровня и векторными полями.

Рассмотрены построение графиков с двумя вертикальными осями, настройка макетов подграфиков и отображение неопределённости.

Изучен принцип создания анимации как последовательности кадров. Реализованы функции построения гипоциклоид и эпициклоид.

Цель освоения основных средств визуализации достигнута. Для полного завершения всех пунктов необходимо устранить отмеченные ограничения и дополнить отсутствующие результаты.

Список литературы

1. Кулябов Д. С. и др. Компьютерный практикум на языке Julia. Лабораторная работа №5 «Построение графиков». Методические указания. С. 37–64.

A Основные фрагменты кода

Листинги команд и результаты их выполнения представлены на скриншотах в основной части отчёта. Ниже приведены основные конструкции, использованные в работе.

A.1 Построение и сохранение графика

```
using Plots

f(x) = (3*x^2 + 6*x - 9) * exp(-0.3*x)

x = collect(range(-5, 10, length=151))
y = f.(x)

p = plot(
    x, y,
    xlabel="x",
    ylabel="f(x)",
    label="f(x)",
    linewidth=2
)
```

```
savefig(p, "function_graph.png")
```

A.2 Приближение полиномом

```
A = [ones(length(x)) x x.^2 x.^3 x.^4 x.^5]

c = A \ y

y_fit = c[1] .+ c[2] .* x .+ c[3] .* x.^2 .+
        c[4] .* x.^3 .+ c[5] .* x.^4 .+ c[6] .* x.^5
```

A.3 Градиент функции

```
h(x, y) = x^3 - 3*x + y^2

x_grid = range(-2, stop=2, length=12)
y_grid = range(-2, stop=2, length=12)

px = [a for a in x_grid for b in y_grid]
py = [b for a in x_grid for b in y_grid]

u = (3 .* px.^2 .- 3) ./ 25
v = (2 .* py) ./ 25
```

A.4 Общий принцип создания анимации

```
x_sin = collect(range(0, stop=2*pi, length=120))
y_sin = sin.(x_sin)

anim = @animate for j in eachindex(x_sin)
    plot(
        x_sin[1:j], y_sin[1:j],
        xlims=(0, 2*pi),
        ylims=(-1.2, 1.2),
        xlabel="x",
        ylabel="sin(x)",
        label="sin(x)"
    )
end

gif(anim, "sine.gif", fps=20)
```